

3DGEER: 3D GAUSSIAN RENDERING MADE EXACT AND EFFICIENT FOR GENERIC CAMERAS

Anonymous authors
Anonymous submission

ABSTRACT

Although 3D Gaussian Splatting (3DGS) (Kerbl et al., 2023) attains a compelling equilibrium between rendering fidelity and computational efficiency, its reliance on approximating three-dimensional Gaussians through their two-dimensional projections constitutes an assumption whose accuracy deteriorates markedly under generic, wide field-of-view (FoV) cameras. Notwithstanding a proliferation of recent extensions, no antecedent work has succeeded in reconciling *projective exactness* with *real-time efficiency* for arbitrary camera geometries. To this end, we introduce 3DGEER, a geometrically exact yet computationally efficient framework for Gaussian rendering. Proceeding from first principles, we derive a closed-form expression for the integration of Gaussian density along a ray, thereby enabling both precise forward rendering and differentiable optimization under arbitrary camera models. So as to preserve efficiency, we further introduce the *Particle Bounding Frustum (PBF)*, which furnishes tight ray-Gaussian association while obviating the need for BVH traversal, together with the *Bipolar Equiangular Projection (BEAP)*, which unifies disparate FoV representations, expedites association, and enhances reconstruction quality. Comprehensive experiments spanning both pinhole and fisheye datasets demonstrate that 3DGEER surpasses prior methods across the full complement of metrics, executes $5\times$ faster than extant projective-exact ray-based baselines, and generalizes to wider FoVs unobserved during training—thereby establishing a new state of the art in real-time radiance field rendering.

1 INTRODUCTION

Inaugurated by 3D Gaussian Splatting (3DGS) (Kerbl et al., 2023), the family of volumetric particle rendering methods formulated under the splatting paradigm (Yu et al., 2024a; Yan et al., 2024; Lu et al., 2024; Kheradmand et al., 2024; Charatan et al., 2024; Mallick et al., 2024) has attained considerable prominence, owing to the conjunction of their striking visual fidelity and their high rendering throughput. This efficiency originates fundamentally in EWA splatting (Zwicker et al., 2001), whereby a local affine approximation renders the projection of a three-dimensional Gaussian as a two-dimensional Gaussian upon the image plane. While such an approximation proves adequate within narrow field-of-view (FoV) regimes, it precipitates substantial error in wide-FoV settings, wherein nonlinear distortion drives the true projection to depart appreciably from any symmetric two-dimensional Gaussian—a discrepancy that becomes especially acute in the fisheye imagery routinely encountered across robotics and autonomous driving (Rashed et al., 2021; Courbon et al., 2007; Kumar et al., 2023; Zhang et al., 2016; Yogamani et al., 2019, 2024; Sekkat et al., 2022). Although recent efforts have sought to accommodate differentiable rendering under either specific fisheye camera models (Liao et al., 2024) or more general camera models (Huang et al., 2024b), these approaches remain fundamentally circumscribed by their dependence upon approximations in the two-dimensional projective space, and have thus far failed to furnish an exact closed-form solution.

In a parallel vein, Celarek et al. (2025) contests the very necessity of exact rendering, arguing that inflating the number of Gaussians (up to 1M) suffices to attenuate the projective approximation inherent in splatting. This argument, however, is substantiated only on small object-centric scenes. Within large-scale scene reconstruction, the requisite Gaussian budget fluctuates considerably across objects and frustums, whereby indiscriminate scaling to larger scenes becomes prohibitively expen-

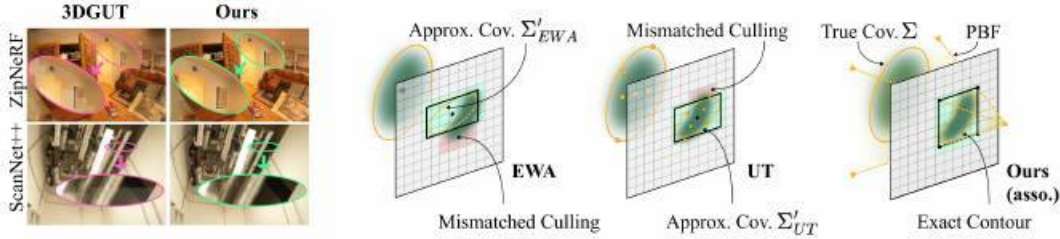


Figure 1: **Linear Approximation Error in Ray-Particle Association.** (Left) Grid-line artifacts precipitated by inaccurate UT association. (Right) A diagram elucidating our association scheme in juxtaposition with competing alternatives. Our method dispenses with the intermediate conic approximation, instead computing the exact bounding structure directly from the true three-dimensional covariance.

sive. Moreover, we demonstrate that the projective errors incurred under generic distorted cameras are substantially larger, to the extent that merely augmenting the Gaussian count cannot bridge the ensuing quality gap (see Fig. 5).

A concurrent line of inquiry confronts volumetric particle rendering directly, casting it within a ray-tracing or ray-marching formulation (Yu et al., 2024b; Condor et al., 2025; Mai et al., 2025; Moënnelocoz et al., 2024; Talegaonkar et al., 2025; Gu et al., 2025). Such formulations accommodate large-FoV camera models as a matter of course and are, by construction, immune to projective approximation error. The attainment of high efficiency, however, persists as a central obstacle—most conspicuously in the task of associating each rendering ray with the 3D particles most pertinent to it. In response, considerable effort has been expended toward its acceleration, exemplified by the adoption of advanced Bounding Volume Hierarchies (BVH) in EVER (Mai et al., 2025) and 3DGRT (Moënnelocoz et al., 2024). Yet, on account of their intrinsic algorithmic complexity and their recalcitrance to parallelization, purely ray-based methods have hitherto failed to rival the efficiency of 3DGS. As an alternative, a hybrid strategy has been advanced in 3DGUT (Wu et al., 2025), which couples ray-tracing rendering with rasterization so as to resolve ray-particle association efficiently. The projective approximation error inherited from the association stage nonetheless continues to compromise exactness in projective geometry and, indeed, risks the emergence of grid-line artifacts (see Fig. 1 Left). To date, then, no extant method has succeeded in conjoining the geometric exactness of ray-based rendering with the high efficiency of splatting.

In the present work, we propose 3DGEER—a novel 3D Gaussian Exact and Efficient Rendering method that secures *exactness in projective geometry* (whose formal definition we furnish in Sec. A) while simultaneously sustaining *high efficiency comparable to that of splatting* for generic cameras, encompassing both perspective geometries and a diversity of fisheye projection models. 3DGEER rests upon three principal contributions: (i) We revisit, from first principles, the heuristic maximum-response function (Keselman & Hebert, 2022) of Gaussians and expose its hitherto implicit connection to projective exactness; building upon this insight, we proceed to derive a differentiable rendering framework endowed with numerical stability. (ii) So as to effect efficient ray-particle association without forfeiting geometric exactness, we recast the problem as one of associating Camera Sub-frustums (CSF) with Particle Bounding Frustums (PBF)—an abstraction analogous to the tile-AABB mapping employed in 3DGS. For each three-dimensional Gaussian, we derive a closed-form solution for the PBF that bounds it tightly, thereby rendering the association highly efficient. (iii) In addition, we propose to sample rays uniformly within a novel Bipolar Equiangular Projection (BEAP) space for the application of color supervision. This sampling strategy not only harmonizes the image-space partitioning with the underlying CSF but also, in point of reconstruction quality, improves upon conventional pinhole and equidistant projections.

Extensive experimentation across both fisheye and pinhole datasets—among them ScanNet++ (Yeshwanth et al., 2023), Zip-NeRF (Barron et al., 2023), Aria (Lv et al., 2025), and MipNeRF (Barron et al., 2021)—establishes that 3DGEER consistently outperforms both prior and concurrent methods, realizing PSNR gains of 0.9–4.8 dB under demanding wide-FoV conditions and of 0.3–2.0 dB on pinhole views. Remarkably, our approach surpasses splat-based methods on wide-FoV cameras even when it is trained exclusively on narrow-FoV data, a fact that underscores its pronounced capacity for generalization to rendering at extreme fields of view.

Table 1: **Summary of Particle Rendering Methods w.r.t. Projective Exactness.** The upper portion enumerates splatting-based methods, whereas the lower portion presents ray-based approaches—a subset of which continue to depend upon projectively approximated association (e.g., EWA or the Unscented Transform).

Method	Render	Particle Asso. Method	Scope	Particle Type	Large FoV	High FPS	Exact Meth.
3DGS (Kerbl et al. 2023)	Splat.	EWA	Img.	3D Gauss.	✗	✓	✗
FisheyeGS (Liao et al. 2024)	Splat.	EWA	Img.	3D Gauss.	✓	✓	✗
GS++ (Huang et al. 2024b)	Splat.	EWA	Img.	3D Gauss.	✗	✓	✗
2DGS (Huang et al. 2024a)	Ray	Splat Bound.	Img.	2D Gauss.	✗	✓	✓
GOF (Yu et al. 2024b)	Ray	EWA	Img.	3D Gauss.	✗	✗	✗
EVER (Mai et al. 2025)	Ray	BVH	Scene	3D Ellip.	✓	✗	✓
3DGRT (Moënné-Loccoz et al. 2024)	Ray	BVH	Scene	3D Gauss.	✓	✓	✓
3DGUT (Wu et al. 2025)	Ray	UT	Img.	3D Gauss.	✓	✓	✗
HTGS (Hahlbohm et al. 2025)	Ray	Splat Bound.	Img.	3D Gauss.	✗	✓	✓
3DGEER (Ours)	Ray	PBF	Frust.	3D Gauss.	✓	✓	✓

2 RELATED WORKS

2.1 SPLATTING-BASED GAUSSIAN RENDERING

The preponderance of 3DGS variants (Yu et al. 2024a, Zhang et al. 2024, Rota Bulò et al. 2024, Gao et al. 2025, Zhao et al. 2025, Kerbl et al. 2024, Hou et al. 2025, Liu et al. 2025) inherit the original EWA splatting strategy (Zwicker et al. 2001, Ren et al. 2002), projecting three-dimensional Gaussians onto the image plane and approximating their appearance as two-dimensional conics. In order to accommodate differentiable rendering under large-FoV camera models afflicted by distortion, methods such as FisheyeGS (Liao et al. 2024) and GS++ (Huang et al. 2024b) estimate the Jacobian of the equidistant or spherical projection so as to adjust the two-dimensional mean and covariance of the projected Gaussians. Such splatting-based approaches, however, remain predicated upon a first-order Taylor expansion of a projection function that is highly non-linear. In wide-FoV settings, wherein the higher-order terms cease to be negligible, this predication engenders pronounced approximation error and a concomitant degradation of reconstruction quality.

Fundamentally, the true projection of a three-dimensional Gaussian under a non-linear camera model—be it an ideal pinhole or a distorted projection—is not a symmetric two-dimensional Gaussian. Stated otherwise, any framework that enlists linear projective geometry to approximate transformations that are inherently nonlinear will inevitably introduce approximation errors incapable of complete elimination. Our approach, by contrast, forgoes such projective approximations in their entirety, aggregating Gaussian density directly along rays within a volumetric rendering formulation.

2.2 RAY-BASED GAUSSIAN RENDERING

Keselman & Hebert (2022) introduce the supposition that a ray intersects the Gaussian density function precisely at the point of maximum intensity response. Elaborating upon this notion, 3DGRT (Moënné-Loccoz et al. 2024) appropriates this heuristic intersection to erect a three-dimensional Gaussian ray-tracing framework, one that permits ray-space sampling without recourse to explicit projection and thereby mitigates wide-FoV distortion. 3DGRT, however, incurs substantial computational overhead, inasmuch as its ray-particle association relies upon costly scene-level spatial partitioning structures in the form of BVHs, culminating in low frame rates.

As an alternative, EVER (Mai et al. 2025) supplants the Gaussian representation with simplified ellipsoids of constant density, whereby a closed-form solution for transmittance-weighted ray integration becomes attainable. This substitution, nonetheless, diminishes the expressive capacity of the representation relative to full three-dimensional Gaussians and, owing to its continued reliance upon BVH-based structures, remains encumbered by limited runtime efficiency. 2DGS (Huang et al. 2024a), for its part, substitutes two-dimensional surfels for three-dimensional Gaussians, thereby admitting exact ray-surface intersections. HTGS (Hahlbohm et al. 2025) reinstates the full Gaussian expressions while appropriating the ray-splat association of 2DGS. Yet, because its association remains confined to screen space, it fails to guarantee projective exactness under wider fields of view. Our contribution, by contrast, resides in deriving closed-form bounds together with a complete rendering pipeline directly within a camera-agnostic angular domain (see the details furnished

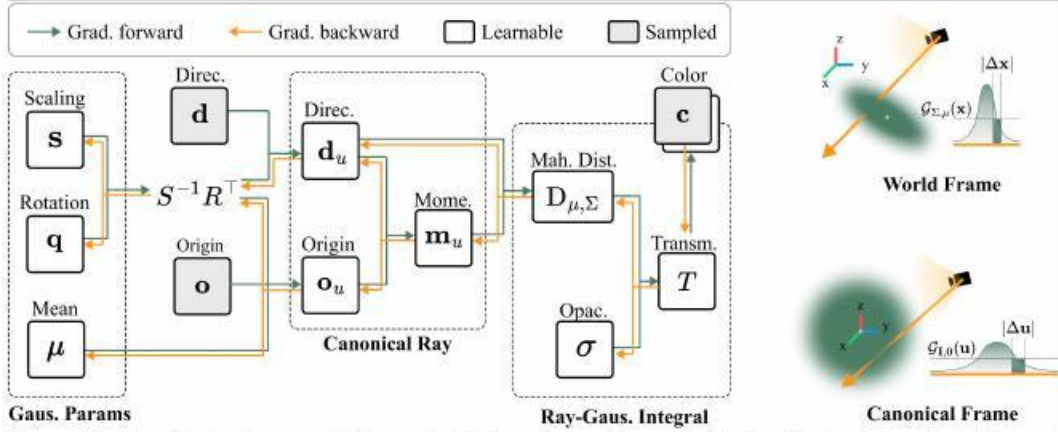


Figure 2: **Gradient Flow and Canonical Transformation.** (Left) An illustration of the forward and backward propagation of gradients, tracing the path from the Gaussian parameters s , q , μ , and the opacity coefficient σ through to the transmittance. (Right) The integral of three-dimensional Gaussian density along a ray, wherein the green box accentuates the product of density and ray-segment length (see Eq. B.5).

in Sec. D; we further devote particular attention, in Sec. D.3 to the relation and the computational distinctions vis-à-vis the parametrization and rendering of the antecedent 2DGS and HTGS.

3DGUT (Wu et al., 2025), a successor to 3DGRT, elevates runtime efficiency by embracing splatting-based particle association, thereby attaining frame rates commensurate with those of 3DGS. Its acceleration, however, is contingent upon the Unscented Transform (UT), a sampling-based approximation deployed to estimate two-dimensional conics within the projective image space. An analogous limitation besets GOF (Yu et al., 2024b), which applies the EWA device directly so as to fit two-dimensional bounding conics and projective centers for association. Notably, under extreme-FoV conditions, imprecise AABB estimation—and the inaccurate tile-to-AABB associations that ensue—can precipitate conspicuous grid-line artifacts (see Fig. 1 Left).

In contradistinction to these methods, our approach resolves the ray-particle association problem directly at the frustum level in three-dimensional space (see Fig. 1 Right), thereby preserving the geometric exactness of ray-based rendering across arbitrary camera models while retaining rendering speeds commensurate with those of 3DGS. Tab. 1 consolidates the representative methods that endeavor to redress projective approximation error, foregrounding the salient distinctions along several consequential dimensions.

3 METHOD

We propose 3DGEER, a Gaussian rendering framework that attains geometric exactness and real-time efficiency in concert under cameras of arbitrary field of view. Our method redresses the two cardinal bottlenecks that have beset prior work—*projection approximation* in splatting-based rendering and *BVH overhead* in ray-based rendering—by way of three complementary components:

- A ray-based rendering formulation (Sec. 3.1): a closed-form, differentiable solution for the integration of Gaussian density along three-dimensional rays, which expunges the linearization error attendant upon projection-based splatting.
- The Particle Bounding Frustum (PBF) (Sec. 3.2): a novel ray-particle association scheme that supplants BVH traversal with exact frustum-particle tests, thereby conferring accuracy and efficiency alike.
- The Bipolar Equiangular Projection (BEAP) (Sec. 3.3): an image representation that unifies arbitrary FoV camera models, expediting PBF computation while enhancing reconstruction quality.

The Appendix additionally provides: (i) the requisite preliminaries and notation (Sec. B.1); (ii) the complete proof, supplementing Sec. 3.1 that establishes the connection between maximum response and projective exactness (Sec. B); (iii) the full derivations of the backward gradients (Sec. C)

together with their numerical stability; and (iv) the full derivations underpinning geometrically exact association (Sec. [D](#)).

3.1 RENDERING WITH EXACT PROJECTIVE GEOMETRY

A pivotal challenge in geometrically exact Gaussian rendering lies in obtaining a closed-form expression for the integration of density and color along a ray. Our strategy consists in mapping each anisotropic Gaussian in world space into a canonical coordinate system within which all Gaussians become isotropic. This mapping reduces the transmittance integral to a measure-preserving change of variables, whereupon a simple closed-form solution is admitted.

Canonical transformation. For each Gaussian with mean μ , rotation R , and scaling S , we define the mapping

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (1)$$

which carries a canonical coordinate $\mathbf{u}$ into world coordinates $\mathbf{x}$. This relation entails $\Delta \mathbf{x} = RS\Delta \mathbf{u}$ and absorbs the Jacobian determinant into the density measure (see Fig. [2 Right](#)). In consequence, each Gaussian is reduced to the shared isotropic form

$$\mathcal{G}_{\mathbf{I},0}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2}\|\mathbf{u}\|^2\right). \quad (2)$$

Closed-form transmittance. For a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, its canonical form is $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$ with

$$\mathbf{o}_u = S^{-1}R^\top(\mathbf{o} - \mu), \quad \mathbf{d}_u = S^{-1}R^\top\mathbf{d}. \quad (3)$$

The accumulated transmittance is then

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I},0}(\mathbf{o}_u + t\mathbf{d}_u) dt = \sigma \exp\left(-\frac{1}{2}D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d})\right), \quad (4)$$

where

$$D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d}) = \frac{\|\mathbf{o}_u \times \mathbf{d}_u\|^2}{\|\mathbf{d}_u\|^2} \quad (5)$$

is the squared perpendicular (Mahalanobis) distance from the ray to the Gaussian center.

Discussion. This result discloses that the transmittance is contingent solely upon the ray’s distance to the Gaussian within the canonical space, thereby yielding a solution that is at once efficient, geometrically exact, and closed-form. Intriguingly, the resulting expression coincides with the “maximum response” heuristic invoked in prior work ([Keselman & Hebert, 2022](#); [Moënné-Loccoz et al., 2024](#)), though our mathematical derivation lays bare the projective exactness underlying it (see the details in Sec. [B](#)). Furthermore, by adhering to our closed-form derivation and to the corresponding backward gradient chain (see Fig. [2 Left](#) and the details in Sec. [C](#)), we explicitly circumvent the numerical instabilities that afflict prior ray-based work ([Yu et al., 2024b](#)) and obviate the need for redundant post-processing, such as the filtering of degenerate Gaussians.

3.2 RAY-PARTICLE ASSOCIATION WITH EXACT PROJECTIVE GEOMETRY

For any given ray-based Gaussian rendering method, both its ultimate accuracy—particularly under large-FoV camera models—and its runtime efficiency hinge critically upon the ray-particle association strategy, a matter that prior work has left largely unexamined. In order to ascertain which Gaussians contribute to each ray, we streamline the association problem by mapping Camera Sub-Frustums (CSFs) onto Particle Bounding Frustums (PBFs)—the tight frustums that enclose each Gaussian at a prescribed

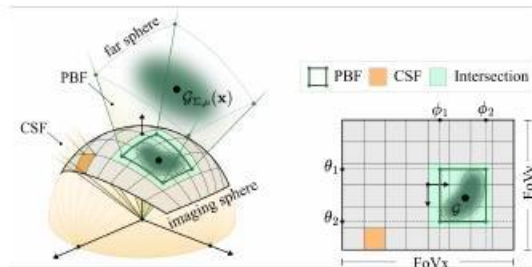


Figure 3: **PBF-CSF Association.** (Left) The PBF, delineated by four planes tangent to a three-dimensional Gaussian ellipsoid. (Right) The intersection of the PBF with the CSF, unfolded onto the BEAP imaging plane.

distance from its center (Fig. 3). A Gaussian is assigned to a CSF whenever its PBF intersects that CSF; all Gaussians thus associated with a CSF are, in turn, deemed contributors to every ray traversing it.

To efficiently compute a PBF, we first define two spherical angles (see Fig. E.1) in the camera space:

$$\theta = \arctan\left(\frac{\mathbf{d}_{c,x}}{\mathbf{d}_{c,z}}\right), \quad \phi = \arctan\left(\frac{\mathbf{d}_{c,y}}{\mathbf{d}_{c,z}}\right), \quad (6)$$

where $(\mathbf{d}_{c,x}, \mathbf{d}_{c,y}, \mathbf{d}_{c,z})$ denotes the normalized direction vector directed toward the camera (view) space point $\mathbf{d}_c$. The angles θ and ϕ correspond, respectively, to the ray's incidence angles within the horizontal (xz) and vertical (yz) planes, each confined to the interval from $-\frac{\pi}{2}$ to $\frac{\pi}{2}$. We define a PBF by means of four planes—two aligned with the θ angle and two with the ϕ angle (see Fig. 3):

$$\mathbf{g}_{\theta_{1,2}} = (-1, 0, \tan \theta_{1,2}, 0)^\top, \quad \mathbf{g}_{\phi_{1,2}} = (0, -1, \tan \phi_{1,2}, 0)^\top, \quad (7)$$

where $\theta_1 < \theta < \theta_2$ and $\phi_1 < \phi < \phi_2$ specify the angular limits.

Given a camera characterized by the extrinsic matrix $V = [R_c | \mathbf{t}_c] \in \text{SE}(3)$ and a canonical-to-world transformation matrix H (see Eq. B.4), we apply the inverse transpose of the camera-to-canonical matrix $(VH)^\top$ so as to parameterize the corresponding canonical planes in terms of the angular limits prescribed in view space. This transformation permits us to derive the constraints governing the PBFs with efficacy. Taking the bounds of θ as an exemplar, the canonical plane is given by:

$$\mathbf{g}_{u_{1,2}} = (VH)^\top \mathbf{g}_{\theta_{1,2}} = -(VH)_0 + \tan \theta_{1,2} (VH)_2, \quad (8)$$

where $(VH)_i$ denotes the i -th row of the matrix, expressed as a column vector.

Positing a Gaussian ellipsoid delimited by a λ -standard-deviation contour, we compute the angular bounds of the local frustum by solving the following quadratic equations, which impose the requirement that the transformed planes be tangent to the ellipsoidal contour:

$$((\lambda^2, \lambda^2, \lambda^2, -1) \circ \mathbf{g}_u^\top) \cdot \mathbf{g}_u = 0, \quad (9)$$

where $\circ$ denotes the Hadamard (element-wise) product, and λ constitutes a parameter governing the spread of the contour, conventionally fixed at 3. An identical constraint is imposed upon $\mathbf{g}_v^\top$ so as to compute the corresponding angular bounds $\phi_{1,2}$. The roots of these quadratic equations determine the tangent values of the angular bounds $[\theta_1, \theta_2]$ and $[\phi_1, \phi_2]$, which in turn define the PBF angular limits. It merits emphasis that solving these quadratic equations subject to the canonical-space constraints honors the true three-dimensional covariance Σ_c and three-dimensional mean μ_c of the Gaussian within camera space. Specifically, for the bounds of θ , the quantity $c = \tan \theta_{1,2}$ satisfies:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad \text{where} \quad \mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\mu_c \mu_c^\top)^{i,j}. \quad (10)$$

This formulation admits of efficient solution (see the details in Sec. D.1) and dispenses both with the intermediate conic approximations characteristic of EWA or UT and with the screen-space constraints that curtail the FoV of HTGS and 2DGS.

3.3 BIPOLAR EQUIANGULAR PROJECTION (BEAP)

We introduce the novel BEAP image representation for a constellation of reasons. First, it represents imagery from large-FoV cameras faithfully, without incurring any loss of field of view. Second, it endows image tiles and their corresponding CSFs with an identical parameterization, expressed through two θ and two ϕ angles, thereby enabling highly efficient PBF association and optimal GPU parallelization. Third, the ray sampling it induces achieves a distribution in three-dimensional space more

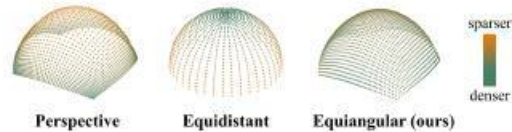


Figure 4: **Comparison of Ray Distributions under Varying Projections.** We project pixels from three imaging spaces onto the unit sphere in order to juxtapose their ray distributions. Among these, our BEAP projection attains the most uniform coverage.

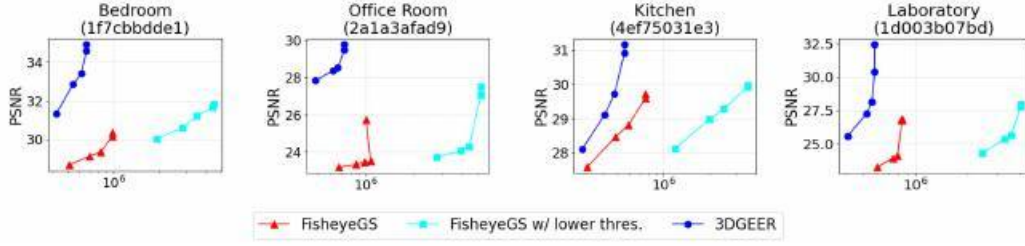


Figure 5: **PSNR $\uparrow$ Trends When Scaling-Up Gaussian Counts.** Brute-force scaling proves unable to close the gap in projective exactness (see the full metrics in Fig. K.3 and Tab. K.3).

uniform than that of conventional projective rendering, which in turn improves training efficiency and reconstruction quality on empirical grounds.

Specifically, we sample the rays uniformly by means of the spherical angular coordinates (θ, ϕ) (see Eq. 6), and thereafter map them back into a discrete $w \times h$ image space via a linear transformation comprising a scaling factor together with a center shift, as follows:

$$(x, y) = \left\lfloor \frac{w\theta}{\text{FoV}_x} + \frac{w+1}{2}, \frac{h\phi}{\text{FoV}_y} + \frac{h+1}{2} \right\rfloor, \quad (11)$$

where $\text{FoV}_x, \text{FoV}_y$ denote the camera’s horizontal and vertical fields of view. As illustrated in Fig. 4 sampling rays uniformly in (θ, ϕ) engenders more balanced spatial coverage throughout the view frustum. Our sampling strategy thus stands in contrast both to projective sampling, which allocates samples disproportionately toward peripheral regions, and to equidistant projection, which oversamples in the vicinity of the image center.

4 EXPERIMENTS

In this section, we subject our proposed method, 3DGEER, to evaluation across a multiplicity of datasets encompassing both pinhole and fisheye camera models. Unless otherwise stated, all experiments are conducted under a default configuration comprising an RTX 4090 GPU and 64GB of RAM. Full implementation details are furnished in Sec. G.

Datasets. We assess performance on generic distorted cameras—the regime in which projective errors prove most consequential—drawing upon all scenes from Fisheye ZipNeRF (Barron et al., 2023), five from ScanNet++ (Yeshwanth et al., 2023), and two from Aria (Lv et al., 2025). The ZipNeRF and ScanNet++ cameras exhibit a 180° diagonal FoV, whereas Aria affords a 110° circular FoV. For pinhole settings, we resort to Pinhole ZipNeRF and MipNeRF360 (Barron et al., 2021).

Evaluation Metrics and Protocol. We report PSNR $\uparrow$, SSIM $\uparrow$, LPIPS $\downarrow$, runtime performance (FPS $\uparrow$), and Gaussian counts $\downarrow$. So as to preclude representation bias, all results are projected back into the image space for full-FoV evaluation upon completion of 30k iterations.

4.1 SCALING GAUSSIANS VS. PROJECTIVE EXACTNESS IN WIDE-FOV CAMERAS

(Celarek et al., 2025) contend that a mere increase in the number of Gaussians (up to 1M) suffices to attenuate the linear approximation gap separating splat-based from ray-based methods. This strategy, however, has been demonstrated only on small object-centric scenes and leaves unaddressed the challenges posed by wide-FoV distortion. To probe the matter further, we scale up the splat-based fisheye method (FisheyeGS) on ScanNet++ scenes by lowering its densification threshold, thereby producing 3.6–5M Gaussians (H200).

As Fig. 5 attests, the red curve (FisheyeGS) and the blue curve (3DGEER) exhibit a pronounced PSNR gap at equal Gaussian counts, betokening an approximation gap in projective exactness (spanning 4k to 30k iterations, beyond which densification typically saturates). The light-blue



Figure 6: **Extreme-FoV Generalization.** 3DGEER reconstructs complete scenes with a smaller Gaussian budget, whereas FisheyeGS succumbs to severe quality degradation.

Table 2: **Comparison on the ScanNet++ Dataset.** Each method is trained either on the full FoV or on the central region alone, and is thereafter evaluated on the full, central, and peripheral regions so as to gauge robustness under varying degrees of distortion. Region-wise metrics are weighted by pixel masks, whereas LPIPS is excluded on account of its not being directly separable by region (full statistics appear in Tab. K.2).

Method	Training FoV	Training Space	Full FoV			Central		Peripheral	
			PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
3DGS	Central	Persp.	-	-	-	31.26	0.948	-	-
FisheyeGS	Full	Eq.	27.81	0.946	0.139	32.44	0.956	23.28	0.914
EVER	Full	KB	29.47	0.924	0.167	29.93	0.924	28.72	0.925
3DGUT	Full	Eq.	30.64	0.944	0.150	31.87	0.945	28.84	0.937
3DGEER (Ours)	Central	BEAP	29.93	0.949	0.130	32.86	0.954	26.64	0.930
3DGEER (Ours)	Full	BEAP	31.50	0.953	0.126	32.64	0.955	28.94	0.945

curve, moreover, traces the scaled-up FisheyeGS: notwithstanding a substantial escalation in memory and computation, its performance saturates at 29.3 PSNR, whereas ours attains 32.1 with a mere 550–700K Gaussians. Full statistics are reported in Tab. K.3

4.2 COMPREHENSIVE EVALUATION ON ALL-FOV NOVEL VIEW SYNTHESIS

We first pit 3DGEER against representative baselines on both fisheye and pinhole benchmarks. In order to render the evaluation comprehensive, we augment it with additional analysis tailored to the idiosyncrasies of each dataset.

ScanNet++. As Tab. 2 demonstrates, 3DGEER consistently outperforms every baseline across all metrics. So as to interrogate the effects of distortion, we evaluate the central and peripheral regions separately (details in Sec. G). Whereas the competing methods perform creditably at the center yet deteriorate toward the periphery, 3DGEER sustains high fidelity throughout, yielding sharper boundaries and fewer artifacts (Fig. K.4). Notably, even when it is trained on central crops alone, 3DGEER continues to surpass FisheyeGS across every region.

Qualitatively (Fig. 6), we appraise extreme-FoV generalization by training on the original dataset FoV (180° diagonal FoV) while testing at appreciably wider FoVs (180° FoV, circular in shape). 3DGEER reconstructs complete scenes with a smaller Gaussian budget, whereas splatting-based methods such as FisheyeGS degrade markedly toward the periphery (see the additional extreme-FoV visualizations in Fig. K.1–K.2).

ZipNeRF. This dataset furnishes frame-wise aligned pinhole and fisheye views. Beyond the standard fisheye evaluation, we undertake *cross-camera generalization*—training on fisheye and testing on pinhole, and conversely—in order to assess robustness. As Tab. 3 indicates, 3DGEER con-

Table 4: **Quantitative Results on the MipNeRF360 Dataset.** Beyond the quality metrics, FPS is reported with an RTX 4090 serving as the default benchmark. Figures drawn from the original works are annotated with $\dagger$ (measured on RTX 6000 Ada) and $\ddagger$ (measured on RTX 5090).

Mgthod	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$
3DGS	27.21	0.815	0.214	343
EVER	27.51	0.825	0.233	36
3DGRT	27.20	0.818	0.248	52 $\dagger$ / 68 $\ddagger$

Table 3: **Comparison on the ZipNeRF Dataset.** The experiments adhere to a cross-camera generalization protocol, wherein each method is trained upon either Fisheye (FE, 1/8 resolution) or Pinhole (PH, 1/4 resolution) data and evaluated separately against both the Fisheye and Pinhole test sets (full statistics appear in Tab. K.9).

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	FE	PH	FE	PH	FE	PH	FE	PH	FE	PH	FE	PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	23.18	26.44	0.858	0.868	0.211	0.239	19.43	26.61	0.791	0.889	0.247	0.212
EVER	25.17	26.14	0.880	0.851	0.153	0.237	22.90	26.45	0.835	0.862	0.207	0.222
3DGUT	24.77	25.59	0.879	0.804	0.183	0.324	18.61	25.96	0.662	0.841	0.300	0.270
3DGEER (Ours)	26.24	27.62	0.897	0.888	0.140	0.214	23.39	27.61	0.846	0.863	0.209	0.254

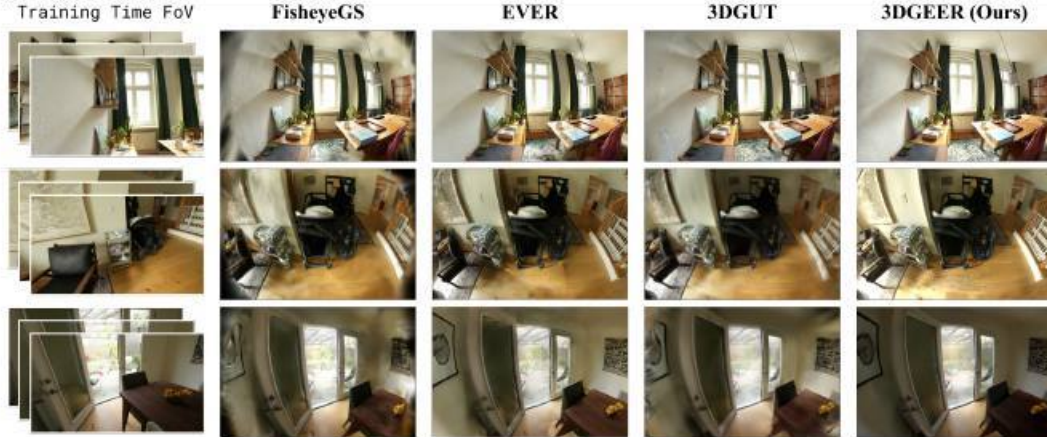


Figure 7: **Qualitative Results on the ZipNeRF Dataset with Cross-Camera Generalization.** The disparities in performance are most pronounced under the exacting Pinhole (PH) train – Fisheye (FE) test configuration, wherein our method evinces a substantial advantage.

sistently outperforms all competing methods across the metrics, exploiting the large-FoV training data to the full through geometrically exact rendering. Even when trained on pinhole data, 3DGEER continues to yield appreciable PSNR gains on both views. FisheyeGS, by contrast, performs well only in the pinhole-to-pinhole regime, and 3DGUT declines precipitously when tested on fisheye subsequent to pinhole training. On the whole, EVER and 3DGEER generalize more favorably by virtue of their geometric exactness, with 3DGEER remaining superior in every case. Qualitatively (Fig. 7), 3DGEER recovers finer detail and exhibits heightened resistance to peripheral artifacts in the most exacting pinhole-to-fisheye setting.

We further evaluate 3DGEER on the **MipNeRF360** dataset under standard narrow-FoV conditions. As Tab. 4 shows, 3DGEER establishes a new state of the art across all the metrics. It runs upwards of $5\times$ faster than exact ray-marching/tracing methods such as EVER and 3DGRT, and stands as the sole projective-exact method whose runtime is commensurate with that of 3DGS. Moreover, it consistently outstrips the projective-approximation baselines (3DGS, 3DGUT) in PSNR, SSIM, and LPIPS. Additional quantitative results on **Aria**—an egocentric fisheye dataset (of moderate 110° FoV, circular in shape) captured under low-light imaging—together with further visualizations, are provided in Sec. J.

4.3 COMPREHENSIVE RUNTIME ANALYSIS

We further analyze runtime on the RTX 4090 by comparing 3DGEER against splatting-based methods (3DGS on **MipNeRF360** and FisheyeGS on **ScanNet++**). Although our ray-based rendering

Table 5: **Training-Time Transmittance Replacement on ScanNet++**. Recourse to splatting-based transmittance inflates the Gaussian count, on account of its larger approximation error, yet the perceptual gap persists relative to our projective-exact formulation (full statistics appear in Tab. K.7).

Transmittance Method	Full FoV			Central		Peripheral		Avg. Gaussian Number (k) ↓
	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	PSNR ↑	SSIM ↑	
3DGS Splats	22.86	0.878	0.177	27.16	0.884	18.56	0.855	1396.4
FisheyeGS Splats	27.90	0.948	0.141	32.49	0.957	23.36	0.917	920.6
Projective-Exact (Ours)	31.50	0.953	0.126	32.64	0.955	28.94	0.945	591.8

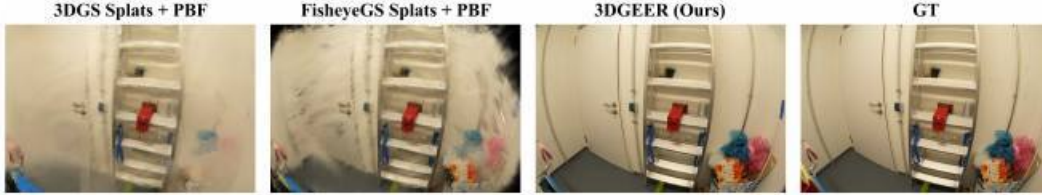


Figure 8: **Rendering-Time Transmittance Func. Replacement**. Substituting the splatting approximation for the projective-exact transmittance at test time alone gives rise to mismatched culling.

incurs a marginally higher cost during the rendering stage, the efficient ray-particle association afforded by PBF substantially curtails the overhead of Gaussian duplication and sorting (see the detailed runtime in Tab. K.4), whereby the overall efficiency comes to rival that of optimized splatting pipelines.

Specifically, PBF furnishes exceedingly tight bounds, associating each tile with a mere ~ 475 Gaussians on average— $3\text{--}5\times$ fewer than EWA or UT (Tab. K.6). This economy accounts directly for the $2.5\text{--}5\times$ runtime acceleration realized in the association stage (Tab. K.5). Projection-based schemes (EWA, UT), by contrast, conservatively overestimate the screen-space extent, thereby precipitating excessive overdraw and inefficient memory access. Even when accelerated by SnugBox (Hanson et al. 2025), they remain hostage to miscentered density approximations, which frequently aggravate artifacts (Fig. I). Beyond considerations of speed, PBF safeguards geometric accuracy and numerical stability, expunging projective approximation error. As Tab. K.6 attests, even when ray-based rendering is applied to Gaussian fields trained under alternative association schemes, PBF remains free of artifacts.

4.4 LOCAL AFFINE APPROXIMATION VS. PROJECTIVE-EXACT INTEGRAL

In order to isolate the effect of our projective-exact transmittance formulation (Eq. 4), we supplant it with the customary local affine splats (i.e., the two-dimensional conic as approximated in 3DGS and FisheyeGS).

Rendering-Time Approx. So as to throw the distinction between the two rendering formulations into sharper relief, we replace the transmittance with the splatting-based approximation at test time alone, retaining all projective-exact components throughout training. As Fig. 8 shows, when the transmittance rests upon the splat approximation while the associations are computed through our exact projective geometry, the inconsistency between them culminates in mismatched culling. This experiment both magnifies and elucidates the gridline artifact documented in 3DGUT (Fig. I), wherein the mismatch arises in the opposite sense—ray-based transmittance conjoined with conic-approximate association.

Training-Time Approx. As Tab. 5 evaluated on ScanNet++, demonstrates, when splatting-based transmittance is invoked during training as well—while association remains projective-exact—the optimizer and densification partially offset both the approximation error and the cross-view inconsistency that splatting introduces. The enlarged loss elicits stronger gradients over the course of training, ultimately occasioning the insertion of a larger ($1.6\text{--}2.4\times$) number of Gaussians to fit the scene. Yet, notwithstanding this augmented Gaussian count, the perceptual gap wrought by the

Table 6: **BEAP vs. Alternative Supervision Schemes.** On ScanNet++, BEAP consistently prevails over competing projections across the full complement of metrics (full statistics appear in Tab. K.8).

Method	Training	Training	Full FoV			Central		Peripheral	
	FoV	Space	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
3DGEER w/o BEAP	Central	Persp.	29.84	0.943	0.131	32.21	0.951	26.23	0.915
3DGEER w/o BEAP	Full	Persp.	21.11	0.853	0.300	21.32	0.855	20.46	0.846
3DGEER w/o BEAP	Full	Eq.	31.05	0.948	0.135	32.21	0.952	28.56	0.935
3DGEER (Ours)	Full	BEAP	31.50	0.953	0.126	32.64	0.955	28.94	0.945

approximation endures, an indication that the error cannot be fully redressed through densification alone.

4.5 BEAP ON/OFF STUDY

We further contrast BEAP supervision with other widely adopted projection models on **ScanNet++**. In this experiment, we hold the projective-exact integral and the PBF fixed for Gaussian transmittance and association, varying solely the ray-sampling distribution employed for supervision (pin-hole, equidistant, and BEAP). As Tab. 6 and Fig. K.5 corroborate, our BEAP-based supervision outperforms every alternative across all metrics. Its more uniform sampling supplies smoother and more consistent gradients, thereby facilitating superior convergence on fine detail.

The discrepancy grows more conspicuous under pinhole cameras. At a fixed resolution, a well-balanced distribution of information may prove more consequential than the mere enlargement of the FoV. Notably, when training within the perspective domain, the full FoV can even be outperformed by the central region (see Tab. 6 rows 1–2), inasmuch as large-FoV images (Fig. K.2 Left) exhibit highly non-uniform angular sampling: informative content concentrates at the center, whereas the less informative periphery, occupying the preponderance of the image area, becomes disproportionately oversampled.

5 CONCLUSIONS

We have presented 3DGEER, a complete, closed-form solution for projective-exact volumetric Gaussian rendering derived from first principles. To complement this formulation, we introduced an efficient method for PBF computation in ray-particle association, one that enables high-speed rendering while upholding exactness in projective geometry. In addition, we proposed the BEAP image representation so as to facilitate effective color supervision under wide-FoV camera models. Extensive experimentation demonstrates that 3DGEER consistently surpasses state-of-the-art methods across a multiplicity of datasets, with results that prove particularly compelling in the demanding wide-FoV regime.

ETHICS STATEMENT

This work is predicated upon publicly available datasets (ScanNet++, ZipNeRF, Aria, and MipNeRF) and implicates neither human subjects, nor private data, nor personally identifiable information. The methods herein developed are intended to advance research in three-dimensional scene reconstruction and rendering, with prospective applications spanning XR, robotics, and autonomous driving. While we anticipate no direct deleterious consequences, we observe that any misuse of high-fidelity three-dimensional reconstruction techniques might occasion privacy concerns; to forestall this, we confine our experiments to research datasets in widespread use throughout the vision community.

REPRODUCIBILITY STATEMENT

We have undertaken considerable efforts to guarantee reproducibility. All mathematical derivations and proofs are furnished in the Appendix (Secs. [B](#), [C](#) and [D](#)). The datasets marshaled in our experiments are publicly available (ScanNet++, ZipNeRF, Aria, and MipNeRF). The particulars of preprocessing, training protocols, and evaluation metrics are delineated in Sec. [G](#). Our implementation, accompanied by training and evaluation scripts, will be released upon publication so as to further facilitate reproducibility.

REFERENCES

- Jonathan T. Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P. Srinivasan. Mip-nerf: A multiscale representation for anti-aliasing neural radiance fields. In *IEEE International Conference on Computer Vision (ICCV)*, 2021.
- Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Zip-nerf: Anti-aliased grid-based neural radiance fields. In *IEEE International Conference on Computer Vision, (ICCV)*, 2023.
- Adam Celarek, George Kopanas, George Drettakis, Michael Wimmer, and Bernhard Kerbl. Does 3d gaussian splatting need accurate volumetric rendering? In *Computer Graphics Forum (CGF)*, 2025.
- David Charatan, Sizhe Lester Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelsplat: 3d gaussian splats from image pairs for scalable generalizable 3d reconstruction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, pp. 19457–19467, 2024.
- Jorge Condor, Sébastien Speierer, Lukas Bode, Aljaz Bozic, Simon Green, Piotr Didyk, and Adrián Jarabo. Don’t splat your gaussians: Volumetric ray-traced primitives for modeling and rendering scattering and emissive media. *ACM Transactions on Graphics (TOG)*, 2025.
- Jonathan Courbon, Youcef Mezouar, Laurent Eckst, and Philippe Martinet. A generic fisheye camera model for robotic applications. In *IEEE/RSJ International Conference on Intelligent Robots and Systems, (IROS)*, pp. 1683–1688, 2007.
- Daniel Duckworth, Peter Hedman, Christian Reiser, Peter Zhizhin, Jean-François Thibert, Mario Lucic, Richard Szeliski, and Jonathan T. Barron. SMERF: streamable memory efficient radiance fields for real-time large-scene exploration. *ACM Transactions on Graphics (TOG)*, 2024.
- Zhongpai Gao, Benjamin Planche, Meng Zheng, Anwesa Choudhuri, Terrence Chen, and Ziyang Wu. 6dgs: Enhanced direction-aware gaussian splatting for volumetric rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Chun Gu, Xiaofei Wei, Zixuan Zeng, Yuxuan Yao, and Li Zhang. Irgs: Inter-reflective gaussian splatting with 2d gaussian ray tracing. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10943–10952, 2025.
- Yuliang Guo, Sparsh Garg, S. Mahdi H. Miangoleh, Xinyu Huang, and Liu Ren. Depth any camera: Zero-shot metric depth estimation from any camera. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

-
- Florian Hahlbohm, Fabian Friederichs, Tim Weyrich, Linus Franke, Moritz Kappel, Susana Castillo, Marc Stamminger, Martin Eisemann, and Marcus Magnor. Efficient perspective-correct 3d gaussian splatting using hybrid transparency. In *Computer Graphics Forum (CGF)*, 2025.
- Alex Hanson, Allen Tu, Geng Lin, Vasu Singla, Matthias Zwicker, and Tom Goldstein. Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 21537–21546, 2025.
- Qiqi Hou, Randall Rauwendaal, Zifeng Li, Hoang Le, Farzad Farhadzadeh, Fatih Porikli, Alexei Bourd, and Amir Said. Sort-free gaussian splatting via weighted sum rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2d gaussian splatting for geometrically accurate radiance fields. In Andres Burbano, Denis Zorin, and Wojciech Jarosz (eds.), *ACM SIGGRAPH Conference Papers*, 2024a.
- Letian Huang, Jiayang Bai, Jie Guo, and Yanwen Guo. Gs++: Error analyzing and optimal gaussian splatting. *CoRR*, 2024b.
- Juho Kannala and Sami S Brandt. A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2006.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (TOG)*, 2023.
- Bernhard Kerbl, Andreas Meuleman, Georgios Kopanas, Michael Wimmer, Alexandre Lanvin, and George Drettakis. A hierarchical 3d gaussian representation for real-time rendering of very large datasets. *ACM Transactions on Graphics (TOG)*, 43(4):1–15, 2024.
- Leonid Keselman and Martial Hebert. Approximate differentiable rendering with algebraic surfaces. In *European Conference on Computer Vision (ECCV)*, 2022.
- Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Yang-Che Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo. *Advances in Neural Information Processing Systems (NeurIPS)*, 37:80965–80986, 2024.
- Varun Ravi Kumar, Ciarán Eising, Christian Witt, and Senthil Kumar Yogamani. Surround-view fisheye camera perception for automated driving: Overview, survey & challenges. *IEEE Transactions on Intelligent Transportation Systems*, 24(4):3638–3659, 2023.
- Zimu Liao, Siyan Chen, Rong Fu, Yi Wang, Zhongling Su, Hao Luo, Li Ma, Linning Xu, Bo Dai, Hengjie Li, et al. Fisheye-gs: Lightweight and extensible gaussian splatting module for fisheye cameras. *ECCV Workshop*, 2024.
- Changkun Liu, Shuai Chen, Yash Sanjay Bhalgat, Siyan HU, Ming Cheng, Zirui Wang, Victor Adrian Prisacariu, and Tristan Braud. Gs-cpr: Efficient camera pose refinement via 3d gaussian splatting. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. Scaffold-gs: Structured 3d gaussians for view-adaptive rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20654–20664, 2024.
- Zhaoyang Lv, Maurizio Monge, Ka Chen, Yufeng Zhu, Michael Goesele, Jakob Engel, Zhao Dong, and Richard Newcombe. Photoreal scene reconstruction from an egocentric device. In *ACM SIGGRAPH Conference Papers*, 2025.
- Alexander Mai, Peter Hedman, George Kopanas, Dor Verbin, David Futschik, Qiangeng Xu, Falko Kuester, Jonathan T Barron, and Yinda Zhang. EVER: exact volumetric ellipsoid rendering for real-time view synthesis. *IEEE International Conference on Computer Vision (ICCV)*, 2025.

- Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De La Torre. Taming 3dgs: High-quality radiance fields with limited resources. In *SIGGRAPH Asia Conference Papers*, 2024.
- Xiaoyang Mao, Lichan Hong, and Arie Kaufman. Splatting of curvilinear volumes. In *Proceedings Visualization '95*, pp. 61–68. IEEE, 1995.
- Christopher Mei and Patrick Rives. Single view point omnidirectional camera calibration from planar grids. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2007.
- Nicolas Moënné-Loccoz, Ashkan Mirzaei, Or Perel, Riccardo de Lutio, Janick Martinez Esturo, Gavriel State, Sanja Fidler, Nicholas Sharp, and Zan Gojcic. 3d gaussian ray tracing: Fast tracing of particle scenes. *ACM Transactions on Graphics (TOG)*, 2024.
- Hazem Rashed, Eslam Mohamed, Ganesh Sistu, Varun Ravi Kumar, Ciaran Eising, Ahmad El-Sallab, and Senthil Yogamani. Generalized object detection on fisheye cameras for autonomous driving: Dataset, representations and baseline. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, (WACV), pp. 2272–2280, 2021.
- Liu Ren, Hanspeter Pfister, and Matthias Zwicker. Object space ewa surface splatting: A hardware accelerated approach to high quality point rendering. In *Computer graphics forum (CGF)*, 2002.
- Samuel Rota Bulò, Lorenzo Porzi, and Peter Kotschieder. Revising densification in gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 347–362. Springer, 2024.
- Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- Ahmed Rida Sekkat, Yohan Dupuis, Varun Ravi Kumar, Hazem Rashed, Senthil Yogamani, Pascal Vasseur, and Paul Honeine. Synwoodscape: Synthetic surround-view fisheye camera dataset for autonomous driving. *IEEE Robotics and Automation Letters (RAL)*, 7(3):8502–8509, 2022.
- Chinmay Talegaonkar, Yash Belhe, Ravi Ramamoorthi, and Nicholas Antipa. Volumetrically consistent 3d gaussian rasterization. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10953–10963, 2025.
- Lee Westover. Footprint evaluation for volume rendering. In *Proceedings of the 17th annual conference on Computer graphics and interactive techniques*, pp. 367–376, 1990.
- Qi Wu, Janick Martinez Esturo, Ashkan Mirzaei, Nicolas Moënné-Loccoz, and Zan Gojcic. 3dgt: Enabling distorted cameras and secondary rays in gaussian splatting. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Zhiwen Yan, Weng Fei Low, Yu Chen, and Gim Hee Lee. Multi-scale 3d gaussian splatting for anti-aliased rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (CVPR), pp. 20923–20931, 2024.
- Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Senthil Yogamani, Ciarán Hughes, Jonathan Horgan, Ganesh Sistu, Padraig Varley, Derek O’Dea, Michal Uricár, Stefan Milz, Martin Simon, Karl Amende, et al. Woodscape: A multi-task, multi-camera fisheye dataset for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9308–9318, 2019.
- Senthil Yogamani, David Unger, Venkatraman Narayanan, and Varun Ravi Kumar. Fisheyebevseg: Surround view fisheye cameras based bird’s-eye view segmentation for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (CVPR), pp. 1331–1334, 2024.
- Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-splatting: Alias-free 3d gaussian splatting. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, (CVPR), pp. 19447–19456, 2024a.

-
- Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian opacity fields: Efficient adaptive surface reconstruction in unbounded scenes. *ACM Transactions on Graphics (TOG)*, 2024b.
- Zheng Zhang, Wenbo Hu, Yixing Lao, Tong He, and Hengshuang Zhao. Pixel-gs: Density control with pixel-aware gradient for 3d gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 326–342. Springer, 2024.
- Zichao Zhang, Henri Rebecq, Christian Forster, and Davide Scaramuzza. Benefit of large field-of-view cameras for visual odometry. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 801–808. IEEE, 2016.
- Hexu Zhao, Haoyang Weng, Daohan Lu, Ang Li, Jinyang Li, Aurojit Panda, and Saining Xie. On scaling up 3d gaussian splatting training. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Ewa volume splatting. In *Proceedings Visualization, 2001. VIS'01.*, pp. 29–538. IEEE, 2001.

CONTENTS

1 Introduction	1
2 Related Works	3
2.1 Splatting-Based Gaussian Rendering	3
2.2 Ray-Based Gaussian Rendering	3
3 Method	4
3.1 Rendering with Exact Projective Geometry	5
3.2 Ray-Particle Association with Exact Projective Geometry	5
3.3 Bipolar Equiangular Projection (BEAP)	6
4 Experiments	7
4.1 Scaling Gaussians vs. Projective Exactness in Wide-FoV Cameras	7
4.2 Comprehensive Evaluation on All-FoV Novel View Synthesis	8
4.3 Comprehensive Runtime Analysis	9
4.4 Local Affine Approximation vs. Projective-Exact Integral	10
4.5 BEAP On/Off Study	11
5 Conclusions	11
a Ethics Statement	12
b Reproducibility Statement	12
c LLM Usage Statement	18
A Appendix: Exactness and Limitation	18
B Appendix: Ray-Gaussian Transmittance with Projective Exactness	18
B.1 Preliminaries And Notations	18
B.2 Differentiable Rendering with Projective Exactness	19
B.3 Ray Integral of Isotropic 3D Gaussian	20
C Appendix: Cookbook of Gradient Computation	21
C.1 Chain Rule	21
C.2 Two-Stage Propagation	21
C.3 Gradients for Canonical Ray	21
C.4 Gradients for Whitening Matrix	22
C.5 Gradients for Scaling and Rotation	22
C.6 Gradients for 3D Mean	23
C.7 Numerical Stability of Our Differentiable Framework	23

864	D Appendix: Additional Details in Association	24
865	D.1 Particle Bounding Frustum (PBF) Solver	24
866	D.2 Details in Computing CSF-PBF Intersection	24
867	D.3 Relation to Prior Works	25
868		
869	E Appendix: BEAP Image Inverse Interpolation	26
870	Pinhole Camera Model	27
871	KB Fisheye Model	27
872		
873	F Appendix: High-level Algorithmic Summary	27
874		
875	G Appendix: Additional Implementation Details	27
876	Baselines	27
877	Implementation Details	27
878	Central and peripheral regions in ScanNet++ evaluation	28
879	Cross-camera generalization evaluation on ZipNeRF	28
880		
881	H Appendix: EWA Splatting Preliminaries	29
882	From camera coordinates to ray coordinates	29
883	Local affine approximation	29
884	Splatting approximation error	29
885		
886	I Appendix: Advantage over Prior Ray-Tracing Methods	30
887		
888	J Appendix: Results on Egocentric Dataset	30
889		
890	K Appendix: Additional Material and Results	32
891	K.1 Additional Material.	32
892	K.2 Additional Results.	32
893		
894		
895		
896		
897		
898		
899		
900		
901		
902		
903		
904		
905		
906		
907		
908		
909		
910		
911		
912		
913		
914		
915		
916		
917		

LLM USAGE STATEMENT

We enlisted a large language model (ChatGPT, OpenAI) in the capacity of a writing assistant. Specifically, the LLM was employed so as to (i) enhance the clarity and readability of the manuscript, (ii) propose stylistic revisions to the figure captions and section transitions, and (iii) furnish alternative phrasings for technical descriptions without altering their scientific content.

The LLM played no part in research ideation, algorithm design, data analysis, or experimental validation. All technical contributions, mathematical derivations, and results are the work of the authors, who assume full responsibility for the content of this paper.

A APPENDIX: EXACTNESS AND LIMITATION

We clarify that our notion of exactness pertains to the *elimination of linear approximation in projective geometry* throughout both the “rendering” and the “ray-particle association” procedures—a regime in which, under our settings, the errors do not diminish as the number of Gaussians increases (see Sec. 4.1). We advance no claim of exactness in the full physical volumetric sense. Our framework continues to entail approximations pertaining to ordering, self-attenuation, and overlap, which constitute the limitations of our approach.

Limitations. Our formulation integrates each Gaussian independently; it does not, in consequence, resolve self-occlusion explicitly, and ordering is dispatched by means of depth-based sorting. As the supplementary videos likewise reveal, this expedient may occasion occasional popping artifacts. To the best of our knowledge, EVER (Mai et al., 2025) stands as the sole method that analytically resolves ordering and self-occlusion, doing so by approximating each Gaussian as a constant-density ellipsoid—though at the cost of a substantial diminution in rendering speed.

Of import here is that prior ray-tracing-based Gaussian renderers such as Fuzzy Metaballs (Kelsman & Hebert, 2022), together with subsequent methods including 3DGR/UT (Moënné-Loccoz et al., 2024; Wu et al., 2025), rest upon the “maximum-response” assumption for ray-Gaussian interaction. Although this assumption inherently disregards ordering and self-attenuation, it has nonetheless been shown to introduce negligible error in practice (see Celarek et al., (2025), Fig. 9-11 / Sec. 7.1), irrespective of the number of Gaussians. This circumstance explains how our depth-based ordering can nevertheless attain the same practical accuracy as extant ray-tracing-based Gaussian renderers.

Future work. Although the practical import of ordering and self-occlusion remains modest in contemporary Gaussian-based ray tracing, these factors fundamentally circumscribe exact volumetric correctness. One auspicious avenue for future inquiry lies in the incorporation of analytic or learnable self-attenuation models—conceivably inspired by ellipsoidal integration (e.g., EVER (Mai et al., 2025)), yet unburdened by heavy runtime cost. A further salient direction concerns the treatment of ordering in a more principled fashion, which may serve to attenuate still further the rare popping artifacts observed under extreme configurations. We hold these limitations to be orthogonal to our contributions on projective exactness, and their resolution would render the framework more robust yet for large-scale or high-dynamic scenes.

B APPENDIX: RAY-GAUSSIAN TRANSMITTANCE WITH PROJECTIVE EXACTNESS

B.1 PRELIMINARIES AND NOTATIONS

We adopt the classical parameterization of three-dimensional Gaussian primitives as introduced in prior work (Zwicker et al., 2001). Each three-dimensional Gaussian distribution is specified by a mean position $\mu \in \mathbb{R}^3$ together with a three-dimensional covariance matrix Σ in the world coordinate system:

$$\mathcal{G}_{\Sigma, \mu}(\mathbf{x}) = \frac{1}{\rho|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^\top \Sigma^{-1}(\mathbf{x} - \mu)\right), \quad (\text{B.1})$$

where we take the normalization constant ρ to be $\sqrt{2\pi}$, and $\mathbf{x} \in \mathbb{R}^3$ denotes a point in world space. This formulation departs from recent methods such as 3DGS (Kerbl et al., 2023) and 3DGUT (Wu et al., 2025), which suppress the determinant term $|\Sigma|^{1/2}$ in the interest of simplifying splatting or association during rendering. Our covariance matrix assumes the form $\Sigma = RSS^\top R^\top$, wherein the scaling matrix $S = \text{diag}(\mathbf{s})$ is constructed from the scaling vector $\mathbf{s} \in \mathbb{R}^3$, and the rotation matrix $R \in \text{SO}(3)$ is derived from a unit quaternion $\mathbf{q} \in \mathbb{R}^4$.

To evaluate the transmittance contributed by a single 3D Gaussian particle along a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, we have:

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\Sigma, \mu}(\mathbf{o} + t\mathbf{d}) dt, \quad (\text{B.2})$$

where σ is the opacity coefficient associated with the Gaussian.

Finally, upon considering a set of three-dimensional Gaussian particles, sorted by depth, that contribute to the rendered color of a given ray, the rendered color C is computed as:

$$C(\mathbf{o}, \mathbf{d}) = \sum_i c_i(\mathbf{o}, \mu_i) T_i \prod_{j=1}^{i-1} (1 - T_j), \quad (\text{B.3})$$

where $c_i(\mathbf{o}, \mu_i)$ yields the view-dependent color of the i -th Gaussian as apprehended from the optical center $\mathbf{o}$, parameterized by spherical harmonics (SH) (Kerbl et al., 2023). By minimizing the reconstruction loss between the rendered and the observed color along each ray, the parameters $\mathbf{s}, \mathbf{q}, \mu$, the coefficient σ , and the SH coefficients are jointly optimized via backpropagation.

B.2 DIFFERENTIABLE RENDERING WITH PROJECTIVE EXACTNESS

The core challenge in achieving projective-exact Gaussian rendering within a ray-marching formulation resides in obtaining a closed-form solution for the integration of Gaussian density and color along a three-dimensional ray. To this end, we first transform the ray—relative to each Gaussian—into a canonical coordinate system in which the transmittance integral (see Eq. B.2) is preserved and the Gaussians become isotropic. We thereupon derive the exact ray color from first principles.

Specifically, we define a canonical coordinate system common to all three-dimensional Gaussian primitives. A point $\mathbf{u} \in \mathbb{R}^3$ within this canonical space is mapped, for each Gaussian, to world coordinates via:

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = H \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (\text{B.4})$$

By virtue of this transformation, an infinitesimal segment $\Delta \mathbf{u}$ along a ray in the canonical frame corresponds to a segment $\Delta \mathbf{x} = RS\Delta \mathbf{u}$ in the world frame.

So as to ensure that the transmittance contributed by each Gaussian along the ray is preserved across coordinate systems, we require the elementary transmittance (i.e., the product of density and segment length) to remain invariant under the transformation (see Fig. 2 Right):

$$\mathcal{G}_{\mathbf{I}, \mathbf{0}}(\mathbf{u}) |\Delta \mathbf{u}| = \mathcal{G}_{\Sigma, \mu}(\mathbf{x}) |\Delta \mathbf{x}|. \quad (\text{B.5})$$

This constraint implies that the Jacobian determinant is subsumed into the measure. Observe that $|\frac{\Delta \mathbf{x}}{\Delta \mathbf{u}}| = |RS| = |\Sigma|^{1/2}$. We may accordingly express the corresponding canonical density as:

$$\mathcal{G}_{\mathbf{I}, \mathbf{0}}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2} \mathbf{u}^\top \mathbf{u}\right), \quad (\text{B.6})$$

which is isotropic and shared across all primitives in canonical space.

This reasoning issues in the significant conclusion that the exact closed-form solution for the transmittance integral of each Gaussian along the ray $\mathbf{r}(t)$ may thus be procured by computing the accumulated density along the transformed ray $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$ within the canonical isotropic Gaussian. Observe that the contribution from $t < 0$ is invariably negligible unless the optical center lies within 3-sigma, in which event all three-dimensional Gaussian rendering frameworks cull the Gaussian during optimization.

This admits a *closed-form* solution expressed in terms of the perpendicular distance from the ray to the Gaussian center, as codified in Theorem 1.

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{o}_u + t \mathbf{d}_u) dt = \sigma \exp\left(-\frac{1}{2} (D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d}))^2\right), \quad (\text{B.7})$$

where $D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d})$ denotes the perpendicular distance from the Gaussian center (that is, the origin within canonical space) to the transformed ray $\mathbf{r}_u(t)$. Intuitively, this quantity likewise represents the minimal *Mahalanobis distance* from any point on the ray to the Gaussian in world space, thereby quantifying the proximity with which the ray passes the particle distribution.

The sole question that remains concerns the manner in which the perpendicular distance from the canonical origin to the transformed ray is to be computed. We derive the direction vector $\mathbf{d}_u$ and the moment vector $\mathbf{m}_u$ of the transformed ray as:

$$\mathbf{d}_u = (S^{-1} R^\top) \mathbf{d}, \quad \mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u, \quad (\text{B.8})$$

where $\mathbf{o}_u = (S^{-1} R^\top) (\mathbf{o} - \boldsymbol{\mu})$ denotes the optical center in canonical space. On this basis, the square of $D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d})$ may be computed efficiently as:

$$(D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d}))^2 = \frac{\mathbf{m}_u^\top \mathbf{m}_u}{\mathbf{d}_u^\top \mathbf{d}_u}. \quad (\text{B.9})$$

By substituting this squared distance into Eq. B.7 and thereafter the transmittance value so derived into Eq. B.3 we obtain the accumulated color along the ray.

B.3 RAY INTEGRAL OF ISOTROPIC 3D GAUSSIAN

Theorem 1. *Given a ray and a standard isotropic 3D Gaussian, the ray-Gaussian integral has a closed-form proportional to:*

$$\exp\left(-\frac{1}{2} D^2\right),$$

where D is the perpendicular distance from the Gaussian center to the ray.

Proof. We first apply to the coordinate system a translation such that the Gaussian $\mathcal{G}$ is centered at the origin, the ray $\mathbf{r}_u$ being translated correspondingly. We then rotate the entire scene so as to align the ray direction with the z -axis. By virtue of the translation-invariance of the integral and the isotropy of the standard Gaussian, these transformations leave the value of the integral unaltered.

The translated standard isotropic 3D Gaussian is defined as:

$$\mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2} \mathbf{u}^\top \mathbf{u}\right), \quad (\text{B.10})$$

which matches Eq. B.6. Let the z -axis aligned ray as:

$$\mathbf{r}_u(t) = \begin{bmatrix} \mathbf{o}_u \\ \mathbf{o}_v \\ 0 \end{bmatrix} + t \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad (\text{B.11})$$

the ray-Gaussian integral becomes:

$$\begin{aligned} \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{r}_u(t)) dt &= \frac{1}{\rho} \exp\left(-\frac{u_0^2 + v_0^2}{2}\right) \int_{t \in \mathbb{R}} \exp\left(-\frac{1}{2} t^2\right) dt \\ &= \frac{\sqrt{2\pi}}{\rho} \exp\left(-\frac{1}{2} D^2\right), \end{aligned} \quad (\text{B.12})$$

where $D^2 = \frac{u_0^2 + v_0^2}{2}$ denotes the squared perpendicular distance from the ray to the Gaussian center in canonical space. The integral is thus proportional to $\exp(-\frac{1}{2} D^2)$, as the theorem asserts. $\square$

Observe that the density function shared by our three-dimensional primitives in the canonical space (see Eq. B.6) conforms to the same formulation as in the isotropic case. When the world-coordinate ray $(\mathbf{o}, \mathbf{d})$ is supplied as the input of D , its value depends in addition upon the world-space Gaussian parameters μ, Σ , which serve to transform the ray into the canonical coordinate system. For each Gaussian contributing to the ray, therefore, we derive the transmittance as:

$$T = \frac{\sqrt{2\pi}\sigma}{\rho} \exp\left(-\frac{1}{2}(D_{\mu,\Sigma}(\mathbf{o}, \mathbf{d}))^2\right), \quad (\text{B.13})$$

which recovers Eq. B.7 upon the choice $\rho = \sqrt{2\pi}$.

C APPENDIX: COOKBOOK OF GRADIENT COMPUTATION

C.1 CHAIN RULE

Recall that $T = \sigma\alpha$ constitutes the ray-Gaussian transmittance (Eq. B.7), wherein σ denotes the opacity coefficient and α the ray-Gaussian integral. We designate $\kappa = D_{\mu,\Sigma}^2$ as the minimal squared Mahalanobis distance from the ray to the Gaussian distribution, S as the Gaussian scaling, and R as the rotation matrix in world space. Furthermore, given $\Sigma = RSS^\top R^\top$ as the three-dimensional covariance matrix of the Gaussian in world space, we take W_{pca} to be the PCA whitening matrix satisfying:

$$W_{\text{pca}} = \Sigma^{-\frac{1}{2}} = S^{-1}R^\top, \quad (\text{C.1})$$

where $S^{-1} = \text{diag}\left(\frac{1}{\mathbf{s}}\right).$

As illustrated in Fig. 2 we may invoke the chain rule to obtain the derivatives w.r.t. the Gaussian parameters, namely the scaling vector $\mathbf{s}$, the rotation quaternion $\mathbf{q}$, and the mean μ :

$$\frac{\partial T}{\partial \mathbf{s}} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{s}}, \quad (\text{C.2})$$

$$\frac{\partial T}{\partial \mathbf{q}} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{q}}, \quad (\text{C.3})$$

$$\frac{\partial T}{\partial \mu} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial \mu}. \quad (\text{C.4})$$

C.2 TWO-STAGE PROPAGATION

In order to curtail the computational overhead of shared propagation, we structure our gradient computation after the two-stage scheme of 3DGS (Kerbl et al. 2023), which comprises `RenderCUDA` (RC) and `PreprocessCUDA` (PC). The former performs ray-wise accumulation, efficiently accelerated by our Particle Bounding Frustum (PBF) association (see Section 3.2), whereas the latter operates with Gaussian-wise complexity, availing itself of view-frustum culling for additional acceleration.

In accordance with Eq. B.7 we may write, respectively, the gradient w.r.t. the opacity coefficient and the shared partial derivative for the Gaussian parameters in RC as:

$$\left.\frac{\partial T}{\partial \sigma}\right|_{\text{RC}} = \alpha, \quad \left.\frac{\partial T}{\partial \kappa}\right|_{\text{RC}} = -\frac{T}{2}. \quad (\text{C.5})$$

The remaining gradients with respect to the Gaussian parameters may then be back-propagated from the squared distance, *e.g.*, $\frac{\partial \kappa}{\partial \mathbf{s}}, \frac{\partial \kappa}{\partial \mathbf{q}}, \frac{\partial \kappa}{\partial \mu}$.

C.3 GRADIENTS FOR CANONICAL RAY

We thereupon backpropagate the gradients to the canonical ray parameters: the origin $\mathbf{o}_u$, the direction $\mathbf{d}_u$, and the moment $\mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u$, as defined in Eq. B.8. These quantities bear directly upon

the Mahalanobis distance in Eq. B.9 and are of critical importance in capturing the canonical ray geometry.

$$\left. \frac{\partial \kappa}{\partial \mathbf{m}_u} \right|_{\text{RC}} = \left(\frac{2}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{m}_u, \quad (\text{C.6})$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{o}_u} \right|_{\text{RC}} = \frac{\partial \kappa}{\partial \mathbf{m}_u} \times \mathbf{d}_u, \quad (\text{C.7})$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{d}_u} \right|_{\text{RC}} = - \left(\frac{2\kappa}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{d}_u + \mathbf{o}_u \times \frac{\partial \kappa}{\partial \mathbf{m}_u}. \quad (\text{C.8})$$

C.4 GRADIENTS FOR WHITENING MATRIX

Recall that $\mathbf{d}_u = W_{\text{pca}} \mathbf{d}$ and $\mathbf{o}_u = W_{\text{pca}}(\mathbf{o} - \boldsymbol{\mu})$, where $\mathbf{o} = -R_c^\top \mathbf{t}_c$ denotes the world-space optical center and $\mathbf{d}$ the ray direction. Once the gradients *w.r.t.* the ray parameters have been obtained, we propagate them further to the PCA whitening matrix W_{pca} by way of outer products with the world-space vectors:

$$\begin{aligned} \text{RenderCUDA: } \left. \frac{\partial \kappa}{\partial W_{\text{pca}}} \right|_{\text{RC}} &= \frac{\partial \kappa}{\partial \mathbf{d}_u} \mathbf{d}^\top; \\ \text{PreprocessCUDA: } \left. \frac{\partial \kappa}{\partial W_{\text{pca}}} \right|_{\text{PC}} &= \frac{\partial \kappa}{\partial \mathbf{o}_u} (\mathbf{o} - \boldsymbol{\mu})^\top; \\ \text{Overall: } \left. \frac{\partial \kappa}{\partial W_{\text{pca}}} \right| &= \left. \frac{\partial \kappa}{\partial W_{\text{pca}}} \right|_{\text{RC}} + \left. \frac{\partial \kappa}{\partial W_{\text{pca}}} \right|_{\text{PC}}. \end{aligned} \quad (\text{C.9})$$

Here, the gradient backpropagated through $\mathbf{d}_u$ is accumulated ray-wise within RenderCUDA, whereas the gradient through $\mathbf{o}_u$ is computed Gaussian-wise within PreprocessCUDA. These two components are subsequently summed so as to yield the complete gradient *w.r.t.* the PCA transformation matrix W_{pca} .

C.5 GRADIENTS FOR SCALING AND ROTATION

Thus, for scaling gradients $\frac{\partial \kappa}{\partial \mathbf{s}} = \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{s}}$, we have:

$$\left. \frac{\partial W_{\text{pca}}^{i,j}}{\partial s_k} \right|_{\text{PC}} = \begin{cases} -\frac{1}{s_k^2} R_{j,k} & \text{if } i = k \\ 0 & \text{otherwise} \end{cases}. \quad (\text{C.10})$$

In order to derive the gradients for rotation $\frac{\partial \kappa}{\partial \mathbf{q}} = \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{q}}$, given the quaternion form $\mathbf{q} = q_r + q_i \mathbf{i} + q_j \mathbf{j} + q_k \mathbf{k}$, we may write the whitening matrix as:

$$W_{\text{pca}} = 2 \begin{pmatrix} \frac{1}{s_1} \left(\frac{1}{2} - q_j^2 - q_k^2 \right) & \frac{1}{s_1} (q_i q_j + q_r q_k) & \frac{1}{s_1} (q_i q_k - q_r q_j) \\ \frac{1}{s_2} (q_i q_j - q_r q_k) & \frac{1}{s_2} \left(\frac{1}{2} - q_i^2 - q_k^2 \right) & \frac{1}{s_2} (q_j q_k + q_r q_i) \\ \frac{1}{s_3} (q_i q_k + q_r q_j) & \frac{1}{s_3} (q_j q_k - q_r q_i) & \frac{1}{s_3} \left(\frac{1}{2} - q_i^2 - q_j^2 \right) \end{pmatrix}. \quad (\text{C.11})$$

In consequence, we derive the following end gradients for the quaternion $\mathbf{q}$:

$$\begin{aligned} \left. \frac{\partial W_{\text{pca}}}{\partial q_r} \right|_{\text{PC}} &= 2 \begin{pmatrix} 0 & \frac{q_k}{s_1} & -\frac{q_j}{s_1} \\ -\frac{q_k}{s_2} & 0 & \frac{q_i}{s_2} \\ \frac{q_j}{s_3} & -\frac{q_i}{s_3} & 0 \end{pmatrix}, \quad \left. \frac{\partial W_{\text{pca}}}{\partial q_i} \right|_{\text{PC}} = 2 \begin{pmatrix} \frac{q_j}{s_1} & -\frac{2q_i}{s_1} & \frac{q_k}{s_1} \\ \frac{q_k}{s_2} & -\frac{q_r}{s_2} & -\frac{2q_j}{s_2} \\ -\frac{2q_k}{s_3} & \frac{q_r}{s_3} & \frac{q_i}{s_3} \end{pmatrix}, \\ \left. \frac{\partial W_{\text{pca}}}{\partial q_j} \right|_{\text{PC}} &= 2 \begin{pmatrix} -\frac{2q_j}{s_1} & \frac{q_i}{s_1} & -\frac{q_r}{s_1} \\ \frac{q_i}{s_2} & 0 & \frac{q_k}{s_2} \\ \frac{q_r}{s_3} & \frac{q_k}{s_3} & -\frac{2q_i}{s_3} \end{pmatrix}, \quad \left. \frac{\partial W_{\text{pca}}}{\partial q_k} \right|_{\text{PC}} = 2 \begin{pmatrix} \frac{q_j}{s_1} & -\frac{2q_i}{s_1} & \frac{q_k}{s_1} \\ -\frac{2q_k}{s_2} & \frac{q_r}{s_2} & \frac{q_i}{s_2} \\ \frac{q_i}{s_3} & \frac{q_j}{s_3} & 0 \end{pmatrix}. \end{aligned} \quad (\text{C.12})$$



Figure C.1: **Artifacts without 3D filtering in GOF.** (Left) the outcome when degenerate Gaussians are left unfiltered in GOF, and (Right) the stable results attained upon applying our forward and backward formulations.

C.6 GRADIENTS FOR 3D MEAN

In parallel, we compute the end gradients *w.r.t.* the Gaussian three-dimensional mean μ as:

$$\left. \frac{\partial \kappa}{\partial \mu} \right|_{\text{PC}} = -W_{\text{pca}}^{\top} \frac{\partial \kappa}{\partial \mathbf{o}_u}. \quad (\text{C.13})$$

C.7 NUMERICAL STABILITY OF OUR DIFFERENTIABLE FRAMEWORK

The choice of forward formulation, of backward gradient computation, and of intermediate cache variables bears critically upon numerical stability. Prior approaches such as GOF (Yu et al., 2024b) are beset by instability in their backward propagation and hence resort to additional three-dimensional filtering heuristics in order to stabilize training. Our closed-form formulation, by contrast, together with its exact gradient chain, forestalls such instability altogether, thereby obviating the need for any auxiliary filtering.

The cardinal distinction of our gradient and rendering pipeline lies in the circumstance that prior methods adopt an alternative expansion of $\mathbf{m}_u^{\top} \mathbf{m}_u$. Ideally—that is, setting numerical considerations aside—the following two forms are geometrically equivalent:

$$\mathbf{m}_u^{\top} \mathbf{m}_u = \|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^{\top} \mathbf{d}_u)^2. \quad (\text{C.14})$$

Whenever a three-dimensional Gaussian possesses one axis scaled toward zero, however, the canonical-space vectors $\mathbf{o}_u$ and $\mathbf{d}_u$ become very nearly parallel. In such a case, $\cos(\angle(\mathbf{o}_u, \mathbf{d}_u)) \rightarrow 1$, while both $\|\mathbf{o}_u\|$ and $\|\mathbf{d}_u\|$ diverge. This state of affairs renders the term $\|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^{\top} \mathbf{d}_u)^2$ highly susceptible to numerical overflow and, in practice, liable to yield negative values, thereby precipitating unstable backward propagation as well.

Fig. C.1 (Left) exhibits the numerical artifacts engendered by GOF. When no additional three-dimensional filtering is applied to expunge degenerate Gaussians, severe artifacts ensue. The snowflake noise originates in numerical overflow within Eq. C.14 wherein negative values are prematurely returned and exceedingly small positive values give rise to amplified Gaussian intensity. Inasmuch as the EWA approximation, endowed with Jacobian and conic, remains numerically stable, the artifacts stem principally from inaccurate maximum-response computation, which likewise induces gridline patterns.

Our CUDA rasterizer implementation, by contrast, computes both the cross product and its gradients directly (Eq. B.9 (forward) and Eqs. C.6–C.9 (backward)), thereby averting this numerical issue at its root. As a result, the forward pass is free of artifacts, while the backward pass yields accurate gradients. Fig. C.1 (Right) illustrates the outcome of applying our forward and backward formulations to GOF: even in the absence of additional three-dimensional filtering to exclude degenerate Gaussians (i.e., flat disk-like or needle Gaussians), our approach effectively eradicates the artifacts occasioned by numerical instability.

D APPENDIX: ADDITIONAL DETAILS IN ASSOCIATION

Section D.1 sets forth our detailed derivation. Section D.2 adduces the implementation considerations requisite for a stable PBF under large-FoV and omnidirectional cameras. Section D.3 elucidates the derivational relationships and states explicitly the conditions under which the antecedent formulas coincide with ours.

D.1 PARTICLE BOUNDING FRUSTUM (PBF) SOLVER

Recalling Eq. 8, we define $P = (VH)^\top \in \mathbb{R}^{4 \times 4}$ as the plane transformation matrix carrying the view space into the canonical space. Taking the unknown bounds for $c = \tan \theta_{1,2}$ as an exemplar, we have:

$$\mathbf{g}_u = -P_0 + cP_2 = \begin{bmatrix} -P_{00} + cP_{02} \\ -P_{10} + cP_{12} \\ -P_{20} + cP_{22} \\ -P_{30} + cP_{32} \end{bmatrix}, \quad (\text{D.1})$$

where P_i denotes i -th row of the matrix, expressed as a 4×1 column vector, and P_{ij} further denotes j -th element. Recall Eq. 9 assuming we are computing the bounding box of the 1-standard-deviation contour ellipsoid of the Gaussian, *i.e.*, $\lambda = 1$, then we have the following quadratic equation:

$$((\mathbf{f} \circ P_2^\top) \cdot P_2) c^2 - 2((\mathbf{f} \circ P_0^\top) \cdot P_2) c + (\mathbf{f} \circ P_0^\top) \cdot P_0 = 0, \quad (\text{D.2})$$

where $\mathbf{f} = (1, 1, 1, -1)$ is a 1×4 row vector. Here we denote the equation as:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad (\text{D.3})$$

where $\mathcal{T}$ is a 3×3 upper triangular matrix defined with $\mathcal{T}_{ij} = (\mathbf{f} \circ P_i^\top) \cdot P_j$.

Similarly, for the bounds of ϕ , we have $c = \tan \phi_{1,2}$ satisfying:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{12}c + \mathcal{T}_{11} = 0. \quad (\text{D.4})$$

Intriguingly, given a camera-view configuration $[R_c \mid \mathbf{t}_c]$, we find that the parameter matrix $\mathcal{T}$ admits of efficient computation from the Gaussian's view-space covariance $\Sigma_c = R_c R S S^\top R^\top R_c^\top$ and three-dimensional mean $\boldsymbol{\mu}_c = R_c \boldsymbol{\mu} + \mathbf{t}_c$:

$$\mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\boldsymbol{\mu}_c \boldsymbol{\mu}_c^\top)^{i,j}. \quad (\text{D.5})$$

We may thus compute the angular bounds—defined by their tangent-space center and extent—from:

$$\begin{aligned} \left(\frac{\tan \theta_1 + \tan \theta_2}{2}, \frac{\tan \phi_1 + \tan \phi_2}{2} \right) &= \left(\frac{\mathcal{T}_{02}}{\mathcal{T}_{22}}, \frac{\mathcal{T}_{12}}{\mathcal{T}_{22}} \right), \\ \|\tan \theta_1 - \tan \theta_2\| &= \frac{2\sqrt{\mathcal{T}_{02}^2 - \mathcal{T}_{22}\mathcal{T}_{00}}}{|\mathcal{T}_{22}|}, \\ \|\tan \phi_1 - \tan \phi_2\| &= \frac{2\sqrt{\mathcal{T}_{12}^2 - \mathcal{T}_{22}\mathcal{T}_{11}}}{|\mathcal{T}_{22}|}. \end{aligned} \quad (\text{D.6})$$

Should the discriminant of either quadratic prove negative—an indication that the camera's optical center lies within the Gaussian ellipsoid and that the angular bounds are consequently undefined—we conservatively clamp the frustum angles to the camera's maximum field of view (FoV).

Observe that this result is computed directly from the true three-dimensional view-space covariance, in contradistinction to methods such as EWA and UT, which lean upon intermediate conic approximations in screen space. Our PBF formulation thus affords exactness and efficiency in equal measure.

D.2 DETAILS IN COMPUTING CSF-PBF INTERSECTION

We sample spherical angles uniformly within the maximum field of view (FoV). The resulting x , y , and z coordinates—that is, the ray direction vectors $\mathbf{d}$ —are projected onto a unit sphere and thereafter transformed into pixel space for inverse color interpolation (see Sec. E). These sampled rays are subsequently grouped according to their corresponding CSFs in preparation for the ensuing Gaussian association.

In the majority of scenes employing a pin-hole camera of limited FoV, the angular bounds within the PBF ordinarily reside in the interval $[-\frac{\pi}{2}, \frac{\pi}{2}]$. In such circumstances, since the tan function increases monotonically over this interval, the intersections between PBF and CSFs may be computed simply by comparing the tan values of their respective angular bounds. In wide-FoV scenarios, however, even when the incidence angle subtended by the Gaussian centers remains within $[-\frac{\pi}{2}, \frac{\pi}{2}]$, the angular bounds employed within the PBF may nonetheless fall beyond the monotonic region of the tan function. Under these conditions, a direct comparison of tan values is liable to yield erroneous PBF-CSF intersections, such as the mistaken pairing of angular bounds as $[\theta'_2, \theta_1]$ (see Fig. D.1 for an illustrative instance).

In order to recover the correct intersections, we introduce a mirror transformation applied to the tangent values of both the PBF angular bounds and the CSF partitions. Drawing inspiration from the CMEI camera model (Mei & Rives 2007), we displace the optical center by means of a mirror parameter ξ , which gives rise to the following transformation:

$$\tan \theta_\xi = \frac{\tan \theta}{1 + \xi \frac{z}{\|z\|} \sqrt{1 + \tan^2 \theta}} = \frac{\sin \theta}{\cos \theta + \xi}, \quad (\text{D.7})$$

where θ_ξ denotes the mirror-transformed angle of θ on the xz -plane under a mirror parameter ξ . We fix $\xi = 1$ throughout all experiments.

Given a pair of tangent values $[\tan \theta_1, \tan \theta_2]$ issuing from our PBF solver (see Eq. D.6), this transformation yields four candidate mirror angles, as illustrated in Fig. D.1. The Gaussian center, by contrast, corresponds to a unique mirror-transformed angle $\theta_{\mu,\xi}$. We exploit this property by selecting the two candidate angles that lie closest to $\theta_{\mu,\xi}$; these, in turn, define the effective lower and upper bounds of the PBF, namely $[\theta_{1,\xi}, \theta_{2,\xi}]$, which are subsequently compared against the CSF partitions so as to compute the correct intersections.

This device secures robust and consistent angular comparisons, even beyond the monotonic domain of the tan function. As our timing analysis reports (Tab. K.5), the pre-processing pipeline—encompassing the mirror-angle-based PBF-CSF intersection computation together with the canonical transformation of all Gaussians—consumes less than 0.13 ms in the aggregate. This figure attests to the efficiency of our approach, even when it is brought to bear upon thousands of Gaussians across wide-FoV scenarios.

D.3 RELATION TO PRIOR WORKS

Substituting the perspective screen space coordinates:

$$u = \tan \theta, \quad v = \tan \phi$$

into our formulation yields expressions identical to those of 2DGS (Huang et al., 2024a) and HTGS (Hahlbohm et al., 2025). This coincidence is to be expected, inasmuch as it constitutes the perspective specialization of our angular-domain formulation.

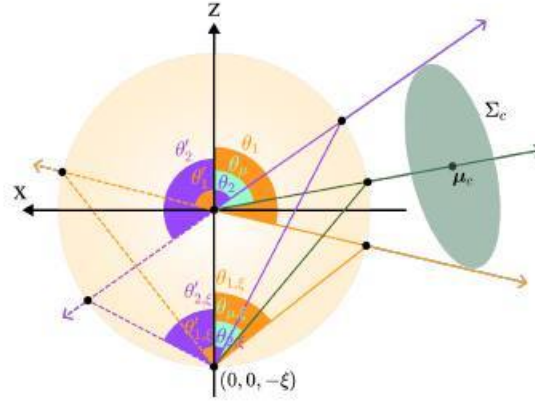


Figure D.1: Mirror Transformation of PBF Angular Bounds. From a given pair of bounds there are generated four candidate mirrored angles $\theta_{1,\xi}, \theta'_{1,\xi}, \theta_{2,\xi}, \theta'_{2,\xi}$. For a Gaussian centered at μ_c , the two lying closest to the Gaussian's corresponding mirrored angle $\theta_{\mu,\xi}$ serve as the PBF bounds, ensuring robust intersection with the mirror-transformed CSF partitions.

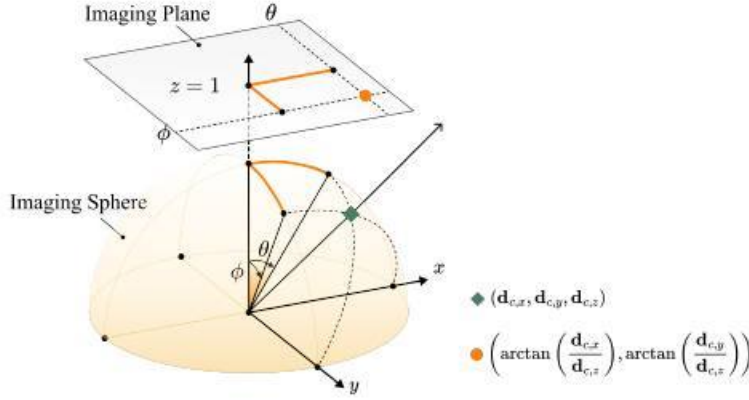


Figure E.1: **Spherical Angles Defining PBF and BEAP.** A view-space direction $\mathbf{d}_c$ is first normalized onto the optical sphere (green spade), whereupon angular projections onto the xz and yz planes are effected. The orange point denotes the resulting coordinate (θ, ϕ) upon the unfolded BEAP imaging plane.

Concretely, 2DGS and HTGS proceed from a screen-space or tangent-plane parameterization and, in consequence, rest upon assumptions valid only for perspective (pinhole) cameras—for instance, geometric identities that materialize when rays are implicitly bound to a planar, z -parallel image plane. Such an assumption holds solely for perspective cameras and does not extend to fisheye or omnidirectional systems, wherein the corresponding surface in screen space is curved. Our derivation, by contrast, commences from a camera-agnostic angular parameterization in three dimensions, parameterizing Camera Sub-Frustums (CSFs) and Particle Bounding Frustums (PBFs) directly. These divergent premises alter the known quantities and constraints available in the course of derivation, and thereby the mathematical conditions under which closed-form bounds and culling guarantees retain their validity for generic cameras.

We would emphasize that mathematics is, in itself, a general instrument; deriving these bounds under weaker assumptions (that is, without recourse to screen-space structure) and corroborating them through comprehensive experiments spanning both the pinhole and the wide-FoV camera families constitutes, however, an essential component of our contribution.

This difference in parameterization bears likewise upon the downstream computational pipeline. Gaussian association, tiling/duplication, and sorting are all contingent upon the manner in which Gaussians map to tiles or bins. Recourse to an angular (CSF-based) domain yields a bucketing structure distinct from that of pinhole screen-space tiling. This distinction is of practical moment for large-FoV scenes: under generic cameras a Gaussian’s bound may appear partially “behind” the camera in screen coordinates (see Fig. D.1), whereby screen-space methods are led to produce mirrored or erroneous tangent-based bounds (the dashed line in Fig. D.1). So as to accommodate these cases robustly, we introduce mirror parameters into the association step (Section D.2) and incorporate the Gaussian center into sorting and correction—components dispensable in perspective-only derivations.

E APPENDIX: BEAP IMAGE INVERSE INTERPOLATION

To prepare the training data, we sample rays so as to cover the original image FoVs comprehensively and supply them as input to our CUDA renderer. Concurrently, we interpolate the ground-truth color for each input ray, thereby ensuring accurate and uniform supervision over the course of training.

The projection of the sampled rays onto the observed images commences with the transformation of the incidence-angle projections θ and ϕ into normalized ray direction vectors $\mathbf{d} = (x, y, z)^\top$, expressed as:

$$x = \frac{\sin \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.1})$$

$$y = \frac{\cos \theta \sin \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.2})$$

$$z = \frac{\cos \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}. \quad (\text{E.3})$$

Several camera models are furnished so as to project the normalized rays $(x, y, z)^\top$ subsequently into pixel space $(x_p, y_p, 1)^\top$.

Pinhole Camera Model. For pinhole camera models, we have in the first instance $\frac{x}{z} = \tan \theta$ and $\frac{y}{z} = \tan \phi$, whence:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.4})$$

KB Fisheye Model. We obtain the incidence angle φ , derived from its projections:

$$r = \sqrt{\tan^2 \theta + \tan^2 \phi} \quad \text{and} \quad \varphi = \arctan(r). \quad (\text{E.5})$$

The fisheye-distorted angle then follows as:

$$\varphi_d = \varphi(1 + k_1\varphi^2 + k_2\varphi^4 + k_3\varphi^6 + k_4\varphi^8). \quad (\text{E.6})$$

In consequence, the rays are projected onto pixel space, whereby inverse color interpolation is rendered possible from:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{f_x \varphi_d}{r} & 0 & c_x \\ 0 & \frac{f_y \varphi_d}{r} & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.7})$$

F APPENDIX: HIGH-LEVEL ALGORITHMIC SUMMARY

We provide a high-level overview of our volumetric rendering pipeline (see Alg. 1 and Alg. 2). For the sake of clarity, we adhere to the structure of 3DGS Kerbl et al. (2023) in order to accentuate the differences: modules newly introduced or supplanted are highlighted in green, and those significantly modified in yellow.

G APPENDIX: ADDITIONAL IMPLEMENTATION DETAILS

Baselines. We compare 3DGEER with the foremost volumetric particle rendering methods that address the projective approximation errors of 3DGS (Kerbl et al. 2023). For large-FoV fisheye inputs, we evaluate against FisheyeGS (Liao et al. 2024), EVER (Mai et al. 2025), and 3DGUT (Wu et al. 2025). Under pinhole settings, we compare in addition with 3DGS and 3DGRT (Moënnec-Loccoz et al. 2024).

Implementation Details. So as to ensure a fair comparison, we align our implementation with the original 3DGS in respect of initialization and hyperparameters on the ScanNet++ and MipNeRF360 datasets. For ZipNeRF, we adopt the training configurations of EVER and SMERF (Duckworth et al. 2024) for all candidate methods. In common with most ray-based frameworks, our method does not access the gradients of `mean2D` directly in order to govern densification or pruning; instead, we optimize by means of the gradients of the view-space `mean3D`. Unless otherwise noted, all models are trained under consistent supervision strategies and architectural settings. For the fish-eye datasets (ScanNet++ and ZipNeRF), we precompute bijective grid mappings (Guo et al. 2025).

ALGORITHM 1: GPU Software Asso. & Render.

Function 3DGEER ($\mu, \mathbf{s}, \mathbf{q}, c, \sigma, \mathbf{r}_c, V$) :

```
CullGaussian( $\mu, V$ ) ▷ Frustum Culling
 $F \leftarrow \text{InitCSFs}(w, h, K)$  ▷ Partition
 $\Sigma_c, \mu_c \leftarrow \text{ViewspaceGaussians}(\mathbf{s}, \mathbf{q}, V)$  ▷ Transform
 $F_{ID} \leftarrow \text{PBF\_Intersect}(\Sigma_c, \mu, F)$  ▷ Sub-frustum Asso.
 $W_{\text{pca}} \leftarrow \text{PCA\_Whitening}(\mathbf{s}, \mathbf{q})$  ▷ Transform
 $\mathcal{G}_{ID}, F_K \leftarrow \text{DuplicateWithKeys}(W_{\text{pca}}, F_{ID})$  ▷ Indices (Gaussian) & Keys (FrustumID+Depth)

SortByKeys( $\mathcal{G}_{ID}, F_K$ ) ▷ Globally Depth Sort
 $R \leftarrow \text{IdentifyFrustumRanges}(F, F_K)$ 
 $I \leftarrow 0$  ▷ Init Canvas (w/ Rays)
for all CSFs  $f \leftarrow I$  do
  for all Rays  $i \leftarrow f$  do
     $r \leftarrow \text{GetFrustumRange}(R, f)$ 
     $\mathbf{m}_u, \mathbf{d}_u \leftarrow \text{Cano\_Ray}(W_{\text{pca}}, \mu, i)$  ▷ Transform
     $D_{\mu, \Sigma} \leftarrow \text{Dist}(\mathbf{m}_u, \mathbf{d}_u)$  ▷ Mah-distance
     $I[i] \leftarrow \text{BlendInOrder}(i, \mathcal{G}_{ID}, r, F_K, D_{\mu, \Sigma}, c, \sigma)$  ▷ Alpha Blending
  end
end
return  $I$ 
```

ALGORITHM 2: 3DGEER Initialization and Training Framework

 w, h : width and height of the training images K, V : intrinsics and extrinsic meta info.

```
 $\mu \leftarrow \text{SfM Points}$  ▷ Init Gaussian Attributes
 $\mathbf{s}, \mathbf{q} \leftarrow \text{InitAttributes}()$  ▷ Position
 $c, \sigma \leftarrow \text{InitCoefficients}()$  ▷ Scaling, Rotation
▷ Color, Opacity
while not converged do
   $\mathbf{r}_c, \hat{C} \leftarrow \text{BEAP}(w, h, K)$  ▷ View Rays / Colors
  ▷ Asso. & Render
   $C \leftarrow \text{3DGEER}(\mu, \mathbf{s}, \mathbf{q}, c, \sigma, \mathbf{r}_c, V)$  ▷ See Alg. 1
   $\mathcal{L} \leftarrow \text{Loss}(C, \hat{C})$  ▷ Loss
   $\mu, \mathbf{s}, \mathbf{q}, c, \sigma \leftarrow \text{Adam}(\nabla \mathcal{L})$  ▷ Backprop & Step
  if IsRefinementIteration( $i$ ) then
     $\mu, \mathbf{s}, \mathbf{q}, c, \sigma \leftarrow \text{Optimize}(\mu, \mathbf{s}, \mathbf{q}, c, \sigma)$  ▷ Kerbl et al. (2023)
  end
end
```

and reproject all rendered outputs into the native KB-Fisheye image space (Kannala & Brandt, 2006) prior to evaluation. Experiments on ScanNet++ and MipNeRF360 are conducted with full-resolution images. For ZipNeRF, on account of the high frame count and the exceedingly large fisheye resolution, we adopt the maximum resolution (see Tab. 3) at which all baselines run reliably on a local desktop configuration.

Central and peripheral regions in ScanNet++ evaluation. The rays sampled within the central region conform to the range

$$\theta \in [-\frac{w}{2f_x}, \frac{w}{2f_x}], \phi \in [-\frac{h}{2f_y}, \frac{h}{2f_y}],$$

while the peripheral region comprises the rays sampled across the remaining angular ranges. Here w, h denote the width and height of the image, and f_x, f_y the focal lengths prescribed by the original KB camera model.

Cross-camera generalization evaluation on ZipNeRF. ZipNeRF furnishes two sets of data: raw fisheye images together with their corresponding undistorted frames, the latter generated by applying distortion correction and cropping the central regions. Although the frames are aligned on a per-frame basis, the camera poses and point clouds were reconstructed independently by means of COLMAP (Schönberger & Frahm, 2016) for each set. Inasmuch as COLMAP’s global optimization refines the camera trajectories independently, the application of a single rigid transformation to align the two pose sets proves insufficient and may introduce alignment errors. So as to secure reliable cross-camera evaluation without incurring errors from structure-from-motion (SfM), we invariably employ the same set of test-time camera poses as those used during training. In the cross-camera experiments, we vary the intrinsic parameters alone while holding the extrinsic poses fixed. This protocol guarantees fair comparisons, unconfounded by the pose inconsistencies that arise from preprocessing.

H APPENDIX: EWA SPLATTING PRELIMINARIES

From camera coordinates to ray coordinates. Following the coordinate conventions of EWA Splatting (Zwicker et al., 2001), we have the following mapping function carrying the camera (view) space points $\mathbf{t}$ to the ray space points $\mathbf{x}$ for the projective transformation in pinhole camera models, i.e., $\tau : \mathbf{t} \rightarrow \mathbf{x}$:

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \tau(\mathbf{t}) = \begin{bmatrix} t_0/t_2 \\ t_1/t_2 \\ \|(t_0, t_1, t_2)^\top\| \end{bmatrix}, \quad (\text{H.1})$$

$$\mathbf{t} = \begin{bmatrix} t_0 \\ t_1 \\ t_2 \end{bmatrix} = \tau^{-1}(\mathbf{x}) = \begin{bmatrix} x_0/l \cdot x_2 \\ x_1/l \cdot x_2 \\ 1/l \cdot x_2 \end{bmatrix}, \quad (\text{H.2})$$

$$\text{, where } l = \|(x_0, x_1, 1)^\top\|. \quad (\text{H.3})$$

These mappings are non-affine, a circumstance that necessitates either computationally expensive (Westover, 1990; Mao et al., 1995) or locally approximated solutions (Zwicker et al., 2001; Kerbl et al., 2023) in order to evaluate the integral in ray space.

Local affine approximation. EWA splatting, due to Zwicker et al. (2001), introduced the local affine approximation, denoted τ_k , so as to retain the linear effects and preserve the k -indexed reconstruction kernels as Gaussians within the two-dimensional screen space. It is defined by the first two terms of the Taylor expansion of τ about the camera-space point $\mathbf{t}_k$; the ray-space point $\mathbf{x}$ in the vicinity of $\mathbf{x}_k$ may thereupon be estimated by:

$$\mathbf{x} = \tau(\mathbf{t}) = \mathbf{x}_k + \mathbf{J}_\tau^{\mathbf{t}_k}(\mathbf{t} - \mathbf{t}_k), \quad (\text{H.4})$$

where $\mathbf{J}_\tau^{\mathbf{t}_k}$ denotes the Jacobian matrix given by the partial derivatives of τ at the point $\mathbf{t}_k$. For the k -indexed three-dimensional Gaussian $\mathcal{G}_{\Sigma, \mu}$ in a given camera coordinate, the approximated density function—obtained through the EWA local affine transformation—may be expressed as a three-dimensional Gaussian $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}$ in the ray coordinate. For $\forall \mathbf{x}$ in the vicinity of $\mathbf{x}_k$, we have:

$$\mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mathbf{x}_k)^\top \Sigma_k^{-1}(\mathbf{x} - \mathbf{x}_k)\right), \quad (\text{H.5})$$

$$\text{where } \Sigma_k = \mathbf{J}_\tau^{\mathbf{t}_k} \Sigma (\mathbf{J}_\tau^{\mathbf{t}_k})^\top. \quad (\text{H.6})$$

The integration of a three-dimensional Gaussian $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}$ along one coordinate axis yields a two-dimensional Gaussian $\mathcal{G}_{\hat{\Sigma}_k, \hat{\mathbf{x}}_k}^{(2)}$; hence:

$$\int_{\mathbb{R}} \mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}(\mathbf{x}) dx_2 = \mathcal{G}_{\hat{\Sigma}_k, \hat{\mathbf{x}}_k}^{(2)}(\hat{\mathbf{x}}), \quad (\text{H.7})$$

where $\hat{\mathbf{x}} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$, the mean $\hat{\mathbf{x}}_k = \begin{bmatrix} x_{k,0} \\ x_{k,1} \end{bmatrix}$, and the 2×2 variance matrix $\hat{\Sigma}_k$ is readily obtained from the 3×3 matrix Σ_k by omitting the third row and column.



Figure I.1: **Qualitative Results on the Stump (Mip-NeRF360).** 3DGEER surpasses both 3DGS-based (Gaussian Splatting) and 3DPRT-based (Particle Ray Tracing) methods on novel view synthesis benchmarks under (Left) MipNeRF360 (pinhole), all the while sustaining high runtime efficiency. The more pronounced differences are discernible within the colored boxes.

Splatting approximation error. Although the density function in ray space cannot be expected to continue obeying the Gaussian distribution once a general projective transformation is applied to the original view space, the exact density may be derived through substitution via Eq. H.2

$$\begin{aligned} \mathcal{G}_{\Sigma, \mu}(\mathbf{t}) &= \mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) \\ &= \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2} (\tau^{-1}(\mathbf{x}) - \mathbf{t}_k)^\top \Sigma_k^{-1} (\tau^{-1}(\mathbf{x}) - \mathbf{t}_k)\right), \end{aligned} \quad (\text{H.8})$$

whereupon the approximation error may be estimated through Monte Carlo integration of the following equation:

$$\mathcal{E}_k = \int_{\mathbb{R}} |\mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) - \mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x})| dx_2. \quad (\text{H.9})$$

Observe that x_2 represents the dimension aligned with the ray direction in the ray coordinate.

I APPENDIX: ADVANTAGE OVER PRIOR RAY-TRACING METHODS

Pure ray-tracing (e.g., 3DGRT) can be projective-exact in the ideal case; in practice, however, the BVH that 3DGRT employs for its intersection tests relies upon an inscribed proxy icosahedron for the sake of greater speed—a proxy tight at the vertices, yet whose face centers, by a property of the icosahedron, lie some $\sim 20\%$ inward. Such proxy shapes may degrade intersection accuracy and, further still, yield inferior accumulated gradients during density control, ultimately eventuating in incorrect geometry (e.g., the missing leaves in Fig. I.1). Moreover, the intersection tests for distant, minute, and cluttered Gaussians are circumscribed by floating-point precision, which occasions noisy rendering. (By way of illustration, in Fig. 15 of the 3DGRT paper, the native rendering in the leftmost block harbors visible speckles across the tree and background areas.) Even 3DGUT, with its UT approximation, outperforms 3DGRT on MipNeRF360. For validation, we deploy 3DGRT’s ray-tracing and rendering engine alongside our 3DGEER rendering pipeline to render from the same pretrained MipNeRF360-stump, and compare on PSNR $\uparrow$ / SSIM $\uparrow$ / LPIPS $\downarrow$: 3DGRT—25.79/ 0.745/ 0.264; ours—26.83/ 0.774/ 0.246 (+1.04/ +0.029/ -0.018). 3DGEER erects a projective-exact and efficient pipeline and, in addition, optimizes it with BEAP color supervision, thereby attaining markedly greater speed together with superior accuracy (Tab. 4).

J APPENDIX: RESULTS ON EGOCENTRIC DATASET

Contemporary egocentric devices typically afford wide FoVs, yet existing pipelines are hard pressed to sustain reconstruction quality when confronted with full-resolution images or when restricted to cropped views. The difficulty is further compounded by low-light imaging conditions, in which the limited light intake—especially with small sensors and aperture-constrained lenses—aggravates the degradation.

We evaluate on the Aria egocentric dataset ($\sim 110^\circ$ FoV, circular in shape). Our method attains an average PSNR of 33.5 dB, consistently surpassing FisheyeGS (32.8 dB) across all metrics (Tab. J.1). Qualitatively, our reconstructions preserve sharper detail in the peripheral regions of the indoor scene



Figure J.1: **Qualitative Results on the Aria Dataset.** Our method yields sharper floor textures in indoor scenes and clearer detail of distant trees in outdoor scenes.

Table J.1: **Quantitative comparison on Aria.** Higher PSNR/SSIM and lower LPIPS are the more favorable.

Method	Scene		PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3DGEER (Ours)	livingroom	(indoor)	36.398	0.933	0.226
	steakhouse	(outdoor)	30.662	0.881	0.246
FisheyeGS	livingroom	(indoor)	35.686	0.927	0.231
	steakhouse	(outdoor)	29.989	0.869	0.261

(Fig. J.1 Left) and recover distant structures such as trees in the outdoor scene (Fig. J.1 Right). The performance gain (+0.7 dB) lies between the improvements observed on pinhole datasets (e.g., MipNeRF360: +0.3 dB; ZipNeRF-cross: +2.0 dB) and those on highly distorted wide-FoV datasets (e.g., ScanNet++/ZipNeRF at 180° diagonal FoV: +0.9–4.8 dB), a placement consonant with Aria’s moderate FoV and level of distortion.

K APPENDIX: ADDITIONAL MATERIAL AND RESULTS

K.1 ADDITIONAL MATERIAL.

Code, video, and an HTML page are provided in the Supplementary Material.

K.2 ADDITIONAL RESULTS.



Figure K.1: **Out-of-Distribution (OOD) Region in Fig. 6** 3DGS collapses to black on account of the numerical pathology $\tan 90^\circ \rightarrow \infty$, whereas our method robustly renders views up to 180° , successfully reconstructing the green chair where splat-based methods (e.g., FisheyeGS) founder. Likewise, in Fig. 6 (zoom-in), our method preserves structures such as the corridor and fridge, table and door, or window and cabinet.



Figure K.2: **Extreme-FoV Generalization.** (Left) A comparison between our method and 3DGS under extreme-FoV pinhole (170°) views (trained on undistorted central regions). (Right) The zoom-out results of Fig. 6, exhibiting the complete circular 180° FoV, wherein our method generalizes robustly to out-of-distribution regions.

Table K.1: **MipNeRF360 Quantitative Results by Sequence.** The aggregate results are presented in Tab. 4

Method	Metric	Bicycle	Bonsai	Counter	Garden	Kitchen	Stump	Flowers	Room	Treehill
Ours	PSNR $\uparrow$	25.63	32.23	29.05	27.60	31.78	26.83	21.83	32.09	22.83
	SSIM $\uparrow$	0.774	0.946	0.910	0.860	0.931	0.774	0.613	0.946	0.637
	LPIPS $\downarrow$	0.223	0.175	0.195	0.136	0.120	0.246	0.340	0.108	0.344

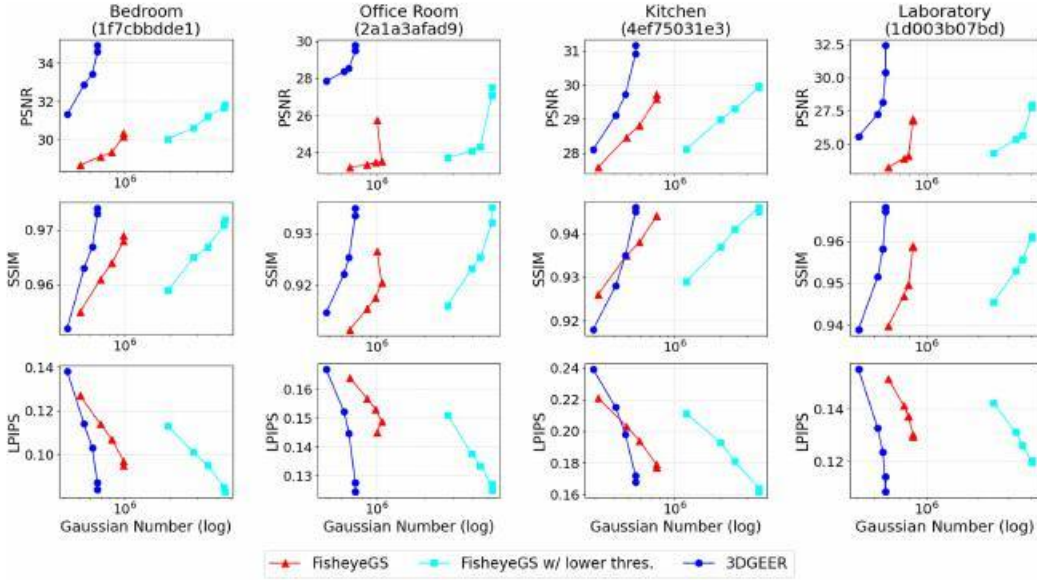


Figure K.3: **PSNR $\uparrow$ SSIM $\uparrow$ LPIPS $\downarrow$ Trends When Scaling-Up Gaussian Counts.** Brute-force scaling proves unable to close the gap in projective exactness.

Table K.2: **ScanNet++ Quantitative Results by Sequence (Full FoV).** The aggregate results are presented in Tab. 2

Scene	EVER			FisheyeGS			3DGUT			3DGEER (Ours)		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Storage	28.24	0.916	0.164	24.74	0.932	0.146	29.26	0.934	0.147	29.18	0.940	0.141
Office Room	27.80	0.899	0.169	25.75	0.927	0.145	29.37	0.928	0.145	29.78	0.935	0.125
Bedroom	32.60	0.948	0.131	31.95	0.970	0.095	34.65	0.967	0.112	34.94	0.974	0.084
Kitchen	29.77	0.924	0.213	29.72	0.944	0.177	30.78	0.941	0.196	31.17	0.946	0.168
Laboratory	28.96	0.936	0.156	26.91	0.959	0.129	29.12	0.948	0.151	32.43	0.968	0.114



Figure K.4: **Qualitative Results on the ScanNet++ Dataset.** The more pronounced differences are discernible within the zoomed-in colored boxes, particularly across structural detail and artifact regions. Our method delivers superior peripheral reconstruction relative to FisheyeGS, exhibits fewer artifacts and sharper detail than EVER, and, in comparison with 3DGUT, produces crisper object boundaries and textures—such as wires, keyboards, and wooden floors.

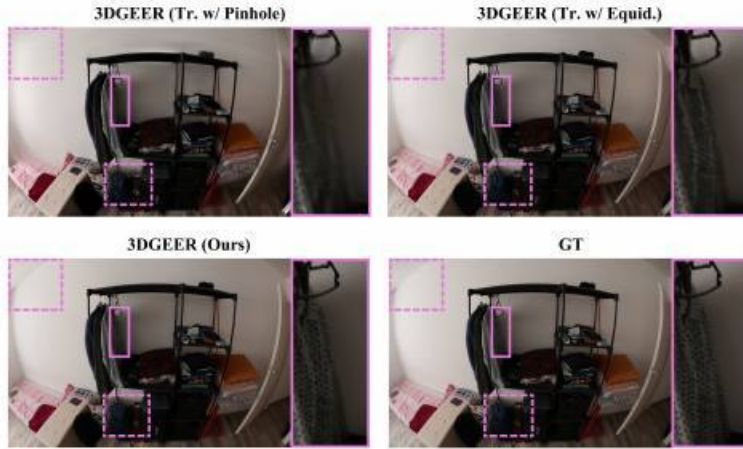


Figure K.5: **Impact of BEAP supervision.** A visual comparison on ScanNet++ reveals that the balanced angular sampling of BEAP yields smoother gradients and a superior preservation of geometric and textural detail relative to pinhole or equidistant sampling. (Full quantitative statistics appear in Tab. [K.8](#))

Table K.3: **Quantitative Results on the ScanNet++**. PSNR $\uparrow$ SSIM $\uparrow$ LPIPS $\downarrow$ trends as the Gaussian counts are scaled up, reported for Bedroom, Office Room, Kitchen, and Laboratory in order.

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	28.67	0.955	0.127	512K	30.03	0.959	0.113	1914K	31.33	0.952	0.138	422K
7k	29.12	0.961	0.114	695K	30.61	0.965	0.101	2829K	32.87	0.963	0.114	543K
10k	29.36	0.964	0.107	827K	31.21	0.967	0.095	3489K	33.42	0.967	0.103	616K
20k	30.17	0.968	0.097	985K	31.68	0.971	0.085	4467K	34.59	0.973	0.087	664K
30k	30.36	0.969	0.095	985K	31.84	0.972	0.083	4535K	34.94	0.974	0.084	664K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	23.19	0.911	0.164	677K	23.72	0.916	0.151	2778K	27.84	0.915	0.167	480K
7k	23.33	0.916	0.157	867K	24.06	0.923	0.138	3936K	28.37	0.922	0.152	621K
10k	23.45	0.918	0.153	983K	24.31	0.925	0.133	4481K	28.54	0.925	0.145	668K
20k	23.51	0.921	0.149	1072K	27.07	0.932	0.127	5272K	29.49	0.933	0.128	732K
30k	25.75	0.927	0.145	1001K	27.50	0.935	0.125	5272K	29.78	0.935	0.125	732K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	27.58	0.926	0.221	314K	28.12	0.929	0.211	1191K	28.11	0.918	0.239	292K
7k	28.47	0.935	0.203	485K	28.99	0.937	0.193	1997K	29.11	0.928	0.215	410K
10k	28.82	0.938	0.194	588K	29.29	0.941	0.181	2491K	29.73	0.935	0.198	476K
20k	29.60	0.944	0.179	760K	29.93	0.946	0.164	3563K	30.91	0.945	0.172	552K
30k	29.72	0.944	0.177	760K	29.97	0.945	0.162	3563K	31.17	0.946	0.168	552K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	23.27	0.940	0.152	602K	24.32	0.945	0.142	2429K	25.59	0.939	0.155	406K
7k	23.94	0.947	0.141	740K	25.36	0.953	0.131	3258K	27.27	0.952	0.133	524K
10k	24.12	0.950	0.137	788K	25.65	0.956	0.126	3569K	28.14	0.958	0.124	562K
20k	26.76	0.959	0.130	838K	27.77	0.961	0.120	4035K	30.39	0.967	0.108	582K
30k	26.91	0.959	0.129	838K	27.97	0.961	0.120	4035K	32.43	0.968	0.114	582K

Table K.4: **Runtime Analysis vs. Other Methods**. 3DGEER is set against efficient splatting-based methods on MipNeRF360 and ScanNet++, by way of a thorough comparison of both stage-wise and total runtime.

Dataset	Method	Avg. Timings (ms) $\downarrow$				
		Prep.	Dup.	Sort	Render	Total
MipNeRF360	3DGS	0.32	0.27	0.65	1.40	2.92
	3DGEER (Ours)	0.37	0.17	0.27	2.10	3.06
ScanNet++	FisheyeGS	0.10	0.63	1.45	2.33	4.70
	3DGEER (Ours)	0.13	0.26	0.59	2.89	3.98

Table K.5: **Runtime Analysis: Asso. Method on ScanNet++**. The efficiency of the PBF-based association is juxtaposed with that of the splatting-based EWA and UT, the acceleration option SnugBox having been integrated into the 3DGEER pipeline.

3DGEER Asso.	Prep.	Avg. Timings (ms) ↓				Total	Avg. FPS ↑
		Dup.	Sort	Render			
EWA	0.13	1.17	3.27	12.69	17.84	56	
+ SnugBox	0.13	0.60	1.42	6.22	8.68	115	
UT	0.13	0.71	1.94	7.97	11.16	90	
+ SnugBox	0.13	0.28	0.64	3.16	4.38	228	
PBF (Ours)	0.13	0.26	0.59	2.89	3.98	251	

Table K.6: **Impact of PBF-CSF Association**. The evaluation avails itself of the artifact-sensitive LPIPS metric. So as to appraise cross-model robustness, each variant is tested in addition upon models trained with the alternative association strategies. We further report the memory cost incurred during rendering, together with the average number of Gaussian associations per tile and the performance gap Δ LPIPS between the in-method and cross-method evaluations, whereby consistency and generalization are quantified.

Asso. Method	Gaus. ↓ per Tile		Mem. ↓ (GB)	LPIPS ↓	Cross-LPIPS ↓	Δ LPIPS ↓ (1e-2)
	mean	std				
EWA	2203	1447	2.2	0.1250	0.1321	0.71
UT	1377	1058	1.4	0.1249	0.1302	0.53
PBF (Ours)	475	427	0.63	0.1245	0.1251	0.06

Table K.7: **Training-Time Transmittance Func. Replacement**. The aggregate results are presented in Tab. 5.

Scene	3DGS Splats + PBF				FisheyeGS Splats + PBF				3DGEER (Ours)			
	PSNR	SSIM	LPIPS	Gaus. Num (k)	PSNR	SSIM	LPIPS	Gaus. Num (k)	PSNR	SSIM	LPIPS	Gaus. Num (k)
Storage	19.27	0.859	0.173	1293	24.75	0.933	0.150	781	29.18	0.940	0.141	429
Office Room	20.89	0.848	0.182	1527	25.89	0.928	0.150	1119	29.78	0.935	0.125	732
Bedroom	27.39	0.923	0.119	1542	31.96	0.970	0.096	1079	34.94	0.974	0.084	664
Kitchen	25.15	0.890	0.239	1241	29.87	0.949	0.182	801	31.17	0.946	0.168	552
Laboratory	21.60	0.869	0.173	1379	27.03	0.962	0.131	823	32.43	0.968	0.114	582

Table K.8: **Impact of Different Supervision Space**. The aggregate results are presented in Tab. 6.

Supervision Space Scene	3DGEER w/o BEAP Central Perspective			3DGEER w/o BEAP Perspective			3DGEER w/o BEAP Equidistant			3DGEER (Ours) BEAP		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Storage	28.42	0.931	0.143	19.74	0.834	0.302	29.03	0.938	0.142	29.18	0.940	0.141
Office Room	28.90	0.927	0.133	21.12	0.825	0.291	29.71	0.932	0.126	29.78	0.935	0.125
Bedroom	33.86	0.965	0.090	24.38	0.882	0.240	34.60	0.973	0.096	34.94	0.974	0.084
Kitchen	30.65	0.941	0.172	23.23	0.889	0.338	31.23	0.941	0.177	31.17	0.946	0.168
Laboratory	27.35	0.951	0.118	17.08	0.834	0.329	30.65	0.956	0.135	32.43	0.968	0.114

Table K.9: **ZipNeRF Quantitative Results by Sequence.** Reported for Alameda, Berlin, London, and NYC in order; the aggregate results are presented in Tab. 3

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	20.74	23.64	0.820	0.802	0.223	0.277	17.39	23.74	0.736	0.839	0.265	0.240
EVER	22.46	23.06	0.852	0.777	0.144	0.274	20.06	22.83	0.789	0.794	0.213	0.250
3DGUT	21.78	22.39	0.841	0.718	0.193	0.368	17.03	22.58	0.588	0.765	0.329	0.311
Ours	24.60	25.00	0.857	0.839	0.154	0.241	21.49	25.21	0.792	0.807	0.232	0.277

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	24.25	28.25	0.895	0.924	0.206	0.209	20.08	27.86	0.848	0.929	0.233	0.194
EVER	25.26	26.97	0.912	0.907	0.155	0.221	23.18	27.72	0.877	0.913	0.196	0.212
3DGUT	25.37	27.02	0.904	0.876	0.197	0.288	18.57	27.43	0.730	0.898	0.297	0.249
Ours	25.51	29.15	0.916	0.925	0.166	0.194	23.58	29.51	0.878	0.915	0.219	0.228

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	23.57	26.65	0.829	0.861	0.253	0.254	20.83	27.33	0.768	0.883	0.288	0.229
EVER	24.59	26.83	0.823	0.859	0.218	0.223	23.14	26.98	0.793	0.860	0.268	0.235
3DGUT	25.61	26.25	0.854	0.803	0.221	0.336	19.82	26.41	0.669	0.833	0.313	0.291
Ours	26.60	27.84	0.895	0.882	0.142	0.230	23.95	27.15	0.840	0.846	0.217	0.289

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	24.15	27.21	0.890	0.885	0.162	0.215	19.41	27.50	0.811	0.907	0.203	0.183
EVER	28.39	27.71	0.931	0.860	0.095	0.228	25.22	28.27	0.879	0.883	0.154	0.192
3DGUT	26.31	26.69	0.917	0.817	0.121	0.302	19.00	27.42	0.659	0.866	0.263	0.228
Ours	28.26	28.49	0.922	0.906	0.097	0.190	24.52	28.55	0.874	0.885	0.168	0.220